<h1 id="CHSH-Inequaliy-Homework">*CHSH Inequaliy Homework*</h1><h3 id="PLEASE-READ-ALL-SECTIONS.-DO-NOT-REMOVE-ANY-MARKDOWN-OR-CODE-CELLS.">**PLEASE READ ALL SECTIONS. DO NOT REMOVE ANY MARKDOWN OR CODE CELLS.**</h3><h2 id="Overview">Overview</h2> In this tutorial, you will run an experiment on a quantum computer to demonstrate the violation of the CHSH inequality. The CHSH (or Bell) Inequality, named after the authors Clauser, Horne, Shimony, and Holt, is used to experimentally prove Bell's theorem (1969). This theorem asserts that local hidden variable theories cannot account for some consequences of entanglement in quantum mechanics. The violation of the CHSH inequality is used to show that quantum mechanics is incompatible with local hidden-variable theories. This is an important experiment for understanding the foundation of quantum mechanics. The 2022 Nobel Prize for Physics was awarded to Alain Aspect, John Clauser and Anton Zeilinger in part for their pioneering work in quantum information science, and in particular, for their experiments with entangled photons demonstrating violation of Bell's inequalities. You can learn more about it in this video <h2 id="Objectives">Objectives</h2>

- Create an open (free) account for IBM Quantum Learning
- Use the Qiskit API to build and run simple circuits
- Use quantum circuit results to justify the violation of the CHSH inequality

<h2 id="Grading">Grading</h2> You are expected to write your own code. It is acceptable to find and use code from the Qiskit API and IBM Quantum Learning webpages, but **it is strictly prohibited to use code obtained from other sources**, especially other students. You are also expected to write descriptive comments explaining what your code is trying to accomplish and why. These are the following gradable items and their weights

1. Building Circuits (40%)
   A. Entangling Qubits (12%)
      - Build Bell Circuit (10%)
      - Draw Bell Circuit (2%)
   B. Adding View Measurements (28%)
      - Building Experiments (20%)
      - Plotting View Circuits (8%)
2. Running Circuits (40%)
   A. Create Plotting Function (5%)
   B. Run Circuit on Ideal Simulator (10%)
   C. Run Circuit on Noisy Simulator (10%)
      - Get a Backend (1%)
      - Creating a Noise Model (2%)
      - Transpile Circuits to Backend (2%)

- Run and Plot Results (5%)

D. Run Circuit on IBM Quantum Computer (15%)

- Parallelize Experiments (7%)
- Transpile Circuit (2%)
- Run Circuit (2%)
- Plot Results (4%)

3. Calculations (20%)

A. Calculating Expectation Values (8%)

- Function to Calculate Expectation Value (1%)
- Calculate Expectation Value for Ideal Simulator (2%)
- Calculate Expectation Value for Noisy Simulator (2%)
- Calculate Expectation Value for IBM Quantum Computer (3%)

B. Calculating CHSH Values (12%)

- Function to Calculate CHSH Value (2%)
- Calculate CHSH Value for Ideal Simulator (3%)
- Calculate CHSH Value for Noisy Simulator (3%)
- Calculate CHSH Value for IBM Quantum Computer (4%)

In [6]:
```
# DO NOT EDIT THIS CELL
n = 20

import sys
print("python", sys.version)

import qiskit
print("qiskit".ljust(n), qiskit.__version__)

import qiskit_aer
print("qiskit_aer".ljust(n), qiskit_aer.__version__)

import qiskit_ibm_runtime
print("qiskit_ibm_runtime".ljust(n), qiskit_ibm_runtime.__version__)

import numpy
print("numpy".ljust(n), numpy.__version__)

import matplotlib
print("matplotlib".ljust(n), matplotlib.__version__)
```

```
python 3.14.7 (tags/v3.14.7:823f032, Aug  5 2026, 10:51:32) [MSC v.1944 64 bit (AMD6
4)]
qiskit              2.5.2
qiskit_aer          0.17.2
qiskit_ibm_runtime  0.50.0
numpy               2.5.3
matplotlib          3.11.2
```

<h3 id="Environment-Verification">Environment Verification</h3> Before you proceed, please verify your kernel is running the **Python 3.12** environment you created with

pip install -r requirements.txt

 If you don't see your environment in

the kernel list, you need to add it using the terminal and relaunch jupyter. You can add your environment to interactive python using this command

 `ipython kernel install --user --name=ENV_NAME` (replacing `ENV_NAME`)

 After you've selected the correct environment, run this next cell, but do NOT edit it in any way! If no errors pop up, then your environment should be able to run this notebook with ease.

<h3 id="Library-Imports">Library Imports</h3> We will need these libraries, classes, and utilities to write and run code for this homework. There should be no import errors if everything is installed correctly.

In [31]:
```
# General
import numpy as np
from typing import Optional

# Plotting routines
import matplotlib.pyplot as plt

# Qiskit imports
from qiskit import QuantumCircuit, QuantumRegister, ClassicalRegister
from qiskit.visualization import plot_histogram
from qiskit.transpiler.preset_passmanagers import generate_preset_pass_manager

# Qiskit Aer imports
from qiskit_aer import AerSimulator
from qiskit_aer.noise import NoiseModel

# Qiskit Runtime imports
from qiskit_ibm_runtime import QiskitRuntimeService, Session, SamplerV2 as Sampler,
```

<h1 id="Section-1:-Building-Circuits-(40%25)">**Section 1: Building Circuits (40%)**</h1>
This section is dedicated to building quantum circuits using Qiskit

<h2 id="Part-1:-Creating-Entangled-Qubits-(12%25)">Part 1: Creating Entangled Qubits (12%)</h2> These experiments will require applying rotations and measurements on entangled qubits, so let's start by writing a Python function that will create and return a 2-qubit 2-cbit circuit that applies a Hadarmard gate on the first qubit and a CNOT targetting the second qubit controlled by the first qubit. An optional barrier can be added to make visualization cleaner.

<h3 id="Building-a-Bell-Circuit-(10%25)">Building a Bell Circuit (10%)</h3> Fill in the Python function `BellCircuit` with Qiskit code that will return a quantum circuit created from `qreg` and `creg` that applies a Hadarmard gate on the first qubit and a CNOT targetting the second qubit controlled by the first qubit. An optional barrier can be added to make visualization cleaner. Do NOT perform any measurements at this stage!

In [32]:
```
# Python function that creates a maximally entangled bell state
def BellCircuit(qreg_name: Optional[str]='q', creg_name: Optional[str]='c') -> Quan
    qreg_q = QuantumRegister(2, qreg_name)
    creg_c = ClassicalRegister(2, creg_name)

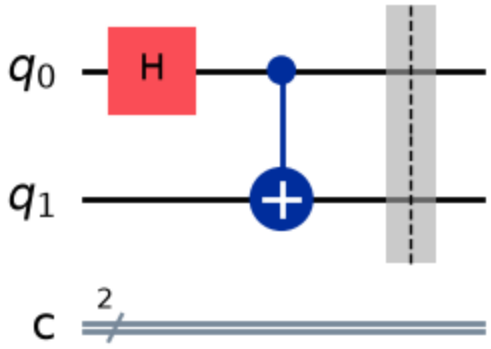
    # creating a bell state circuit
```

```
    bell_circuit = QuantumCircuit(qreg_q, creg_c)  # create a quantum circuit with
    bell_circuit.h(qreg_q[0])                      # apply a Hadamard gate to the f
    bell_circuit.cx(qreg_q[0], qreg_q[1])         # apply a CNOT gate with the firs
    bell_circuit.barrier()                         # add a barrier to separate the
    return bell_circuit

bell = BellCircuit()
```

In [33]:
```
# Visualizing the Bell state circuit
bell.draw('mpl')
```

Out[33]:



<h2 id="Part-2-Adding-View-Measurements-(28%25)">Part 2 Adding View Measurements (28%)</h2> Now that we have a maximally entangled bell state, let us measure it in 4 different views *AB*, *Ab*, *aB*, and *ab*.

- *AB*: Rotate qubit 1 by around the Y-axis by $-\frac{\pi}{4}$ measuring both qubits in the Z basis
- *Ab*: Rotate qubit 1 by around the Y-axis by $\frac{\pi}{4}$ measuring both qubits in the Z basis
- *aB*: Rotate qubit 1 by around the Y-axis by $-\frac{\pi}{4}$ measuring qubit 0 in the X basis and qubit 1 in the Z basis
- *ab*: Rotate qubit 1 by around the Y-axis by $\frac{\pi}{4}$ measuring qubit 0 in the X basis and qubit 1 in the Z basis

<h3 id="Drawing-the-Circuit-(2%25)">Drawing the Circuit (2%)</h3> Use Qiskit API to `draw` the bell circuit using the matplotlib '`mpl`' style

In [36]:
```
# Write code that will create circuits for AB, Ab, aB, and ab views
views = ("AB","Ab","aB","ab")

# Measurement function that takes in a circuit and a view and adds the appropriate
def MeasureWithView(circuit: QuantumCircuit, view: str):
    # YOUR CODE BELOW
    if view[0] == 'a':
        circuit.h(0)
    if view[1] == 'b':
        circuit.h(1)
    circuit.measure_all()

    return circuit
```

```
circuits = [MeasureWithView(BellCircuit('q'+view,'c'+view),view) for view in views]
```

<h3 id="Building-Experiments-(20%25)">Building Experiments (20%)</h3> Fill in the Python function <code>MeasureWithView</code> with Qiskit code that will create these different view circuits leveraging the previous <code>BellCircuit</code> function, then plot them using the given code below.

<h3 id="Plotting-the-Different-View-Circuits-(8%25)">Plotting the Different View Circuits (8%)</h3> Fill in the <code>PlotCircuits</code> Python function with code that will create a figure containing a drawing of each view circuit by calling its <code>draw</code> method. Check that your circuits make sense and appear as described in the instructions.

In [37]:
```
# Plot the circuits into 1 figure
def PlotCircuits(*circuits: QuantumCircuit):
       # Calculate the number of circuits and create a subplot for each circuit
    num_circuits = len(circuits)

    # Create a figure with subplots for each circuit
    fig, axs = plt.subplots(1, num_circuits, figsize=(5 * num_circuits, 5))

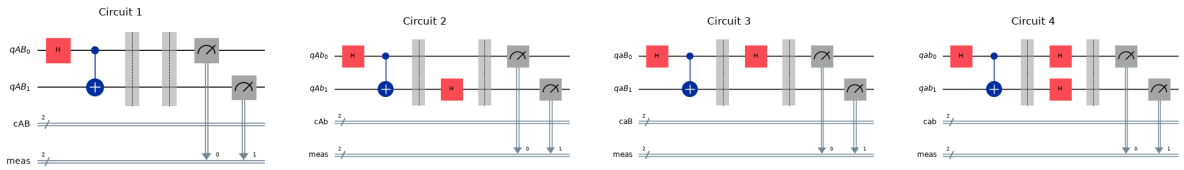
    # If there is only one circuit, axs will not be a list, so we need to make it a
    if num_circuits == 1:
        axs = [axs]

    # Loop through each circuit and draw it on the corresponding subplot
    for i, circuit in enumerate(circuits):
        # Draw directly on the subplot axis
        circuit.draw('mpl', ax=axs[i])
        axs[i].set_title(f'Circuit {i+1}')

    plt.tight_layout()
    plt.show()


    plt.show()

PlotCircuits(*circuits)
```



<h1 id="Section-2:-Running-Circuits-(40%25)">**Section 2: Running Circuits (40%)**</h1> This next section is dedicated to running your quantum circuits on both ideal and noisy simulators, and finally plotting the results.

<h2 id="Part-1:-Creating-a-Plotting-Function-(5%25)">Part 1: Creating a Plotting Function (5%)</h2> Fill the <code>PlotCircuitResults</code> Python function with code that will utilize

`plot_histogram` to visualize the counts of each observable from a circuit job. There is an example dictionary you should test with.

In [38]:
```
# Plot the circuits into 1 figure
def PlotCircuitResults(circuit_results: dict):
      # Calculate the number of results and create a subplot for each result
    num_results = len(circuit_results)
    fig, axs = plt.subplots(1, num_results, figsize=(5 * num_results, 5))

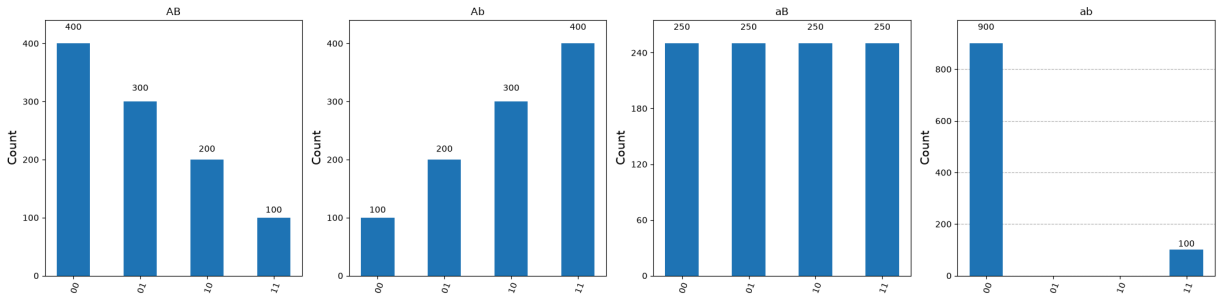
    # If there is only one result, axs will not be a list, so we need to make it a
    if num_results == 1:
        axs = [axs]

    # Loop through each result and plot the histogram on the corresponding subplot
    for i, (key, counts) in enumerate(circuit_results.items()):
        plot_histogram(counts, ax=axs[i])
        axs[i].set_title(key)

    plt.tight_layout()

    plt.show()

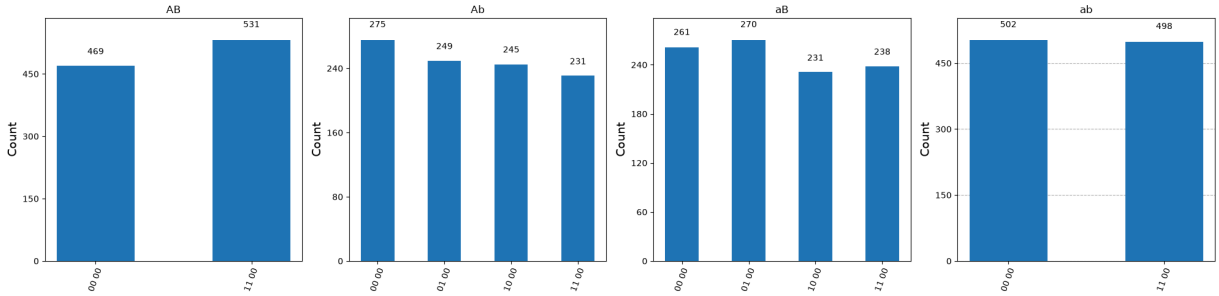
PlotCircuitResults({
    "AB":{"00": 400, "01": 300, "10": 200, "11": 100},
    "Ab":{"00": 100, "01": 200, "10": 300, "11": 400},
    "aB":{"00": 250, "01": 250, "10": 250, "11": 250},
    "ab":{"00": 900, "01": 000, "10": 000, "11": 100},
})
```



<h2 id="Part-2:-Running-Circuits-on-an-Ideal-Simulator-(10%25)">Part 2: Running Circuits on an **Ideal Simulator** (10%)</h2> Use Qiskit's AerSimulator to simulate 1000 shots for each view circuit, then use the `PlotCircuitResults` function to plot the counts from each view circuit job result.

In [39]:
```
# Run the circuits on the Aer simulator and collect the results
simulator = AerSimulator()
results = {}
# Run each circuit on the simulator and store the results in the results dictionary
for circuit, view in zip(circuits, views):
    job = simulator.run(circuit, shots=1000)
    result = job.result()
    counts = result.get_counts()
    results[view] = counts

PlotCircuitResults(results)
```



<h2 id="Part-3:-Running-Circuits-on-an-IBM-Noisy-Simulator-(10%25)">Part 3: Running Circuits on an **IBM Noisy Simulator** (10%)</h2> Use Qiskit's AerSimulator and NoiseModel to simulate 1000-shot runs of each view circuit as if they were ran on an 127-qubit IBM quantum computer, then use the `PlotCircuitResults` function to plot the quasi-dists from each view circuit job result.

<h3 id="Loading-Your-QiskitRuntimeService-Account">Loading Your QiskitRuntimeService Account</h3> To request a backend from IBM, you need an IBM Quantum account synced to this notebook. For a service (like IBM) to track users jobs and utilization of their hardware resources, Qiskit uses an API token, CRN, and instance which are tied to your IBM Quantum account. It is critical to keep it secret because any device could use it and use QPU time under your account which would drain your credits / computing funds. See the QiskitRuntimeService for more info.

In [42]:
```
QiskitRuntimeService.save_account(
    token="_p2Fc5pydLOg6_4tWFXtrbMhOxBiJ2_8zCciIQbKaPPo",
    instance="crn:v1:bluemix:public:quantum-computing:us-east:a/d8cce3288f8442818ca
    set_as_default=True,
    overwrite=True
)
```

Once you run the cell above, you only need to run the cell below to get the service because your API token is now safely stored on your device, but keep in mind if you run this notebook on another device, the saved account may be different.

In [43]:
```
# Load saved credentials
service = QiskitRuntimeService()
```

<h3 id="Getting-a-Backend-(1%25)">Getting a Backend (1%)</h3> Use the QiskitRuntimeService to request a real 127-qubit quantum computer backend from the `ibm_quantum` channel.
PLEASE DO NOT EXPLICITLY WRITE YOUR API TOKEN IN THIS NOTEBOOK, BUT RATHER SAVE YOUR ACCOUNT

In [46]:
```
backend = service.backend("ibm_fez")
print(f"Selected backend: {backend.name}")
```

```
Selected backend: ibm_fez
```

Run the cell below to open a web browser to view the backend you requested

In [47]:
```
import webbrowser
```

```
webbrowser.open(f"https://quantum.cloud.ibm.com/computers?system={backend.name}")
```

Out[47]: True

<h3 id="Creating-a-Noise-Model-(2%25)">Creating a Noise Model (2%)</h3> Create a `NoiseModel` using the backend you just requested, then create an `AerSimulator` using that noise model.

In [49]:
```
# YOUR CODE BELOW
from qiskit_aer import AerSimulator
from qiskit_aer.noise import NoiseModel

# Create the noise model from your backend
noise_model = NoiseModel.from_backend(backend)

# Set up your AerSimulator with the noise model
simulator = AerSimulator(noise_model=noise_model)
```

<h3 id="Transpile-Circuit-for-Noisy-Backend-(2%25)">Transpile Circuit for Noisy Backend (2%)</h3> Use the `general_preset_pass_manager` to create a transpiler for your backend, then transpile your circuits.

In [50]:
```
# YOUR CODE BELOW
from qiskit.transpiler.preset_passmanagers import generate_preset_pass_manager

# Generate the pass manager for your backend with optimization level 1 or 3
pm = generate_preset_pass_manager(optimization_level=1, backend=backend)

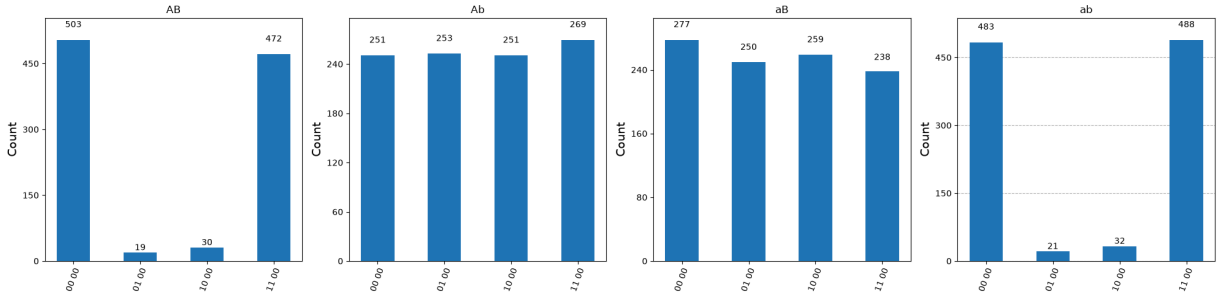
# Transpile your CHSH circuits (replace 'circuits' with the actual variable name of
transpiled_circuits = pm.run(circuits)
```

<h3 id="Run-Transpiled-Circuits-and-Plot-Results-(5%25)">Run Transpiled Circuits and Plot Results (5%)</h3> Run the transpiled circuits using the noisy simulator, then call `PlotCircuitResults` plot the counts.

In [57]:
```
# Run the transpiled circuits on the noisy simulator
job = simulator.run(transpiled_circuits)
result = job.result()

# Convert the list of counts into a dictionary mapped to each view
counts_dict = {views[i]: c for i, c in enumerate(result.get_counts())}

# Plot the results using the dictionary
PlotCircuitResults(counts_dict)
```



<h2 id="Part-4:-Running-Circuits-on-an-IBM-Quantum-Computer-(15%25)">Part 4: Running Circuits on an IBM Quantum Computer (15%)</h2> Running a circuit on IBM's quantum hardware is free but limited by Quantum Processing Unit (QPU) time per month. As such, caution should be taken before sending jobs to a real device. If the job is not set up correctly, it can easily result in a waste of QPU time.

 When a job is sent for execution, it enters a job queue because a QPU can only service one job at a time and thousands of users are submitting jobs daily. QPU time is only accumulated when the job starts running after it has finished queueing.

<h3 id="Optimizing-Circuits-for-Real-Backend">Optimizing Circuits for Real Backend</h3> We have 4 experiments that utilize 2 qubits each, but our real backend uses quantum hardware that supports over 100 qubits. To be more efficient with that computing resource, we should parallelize some shots by using more qubits to save QPU time. To achieve this, we will create a new 8-qubit circuit that runs all 4 experiments simulaneously on disjoint pairs of qubits. This will cut our QPU usage down by 75% since we are effectively parallelizing the experiments. Just run the code below to see how this works.

In [58]:
```
from qiskit import QuantumCircuit

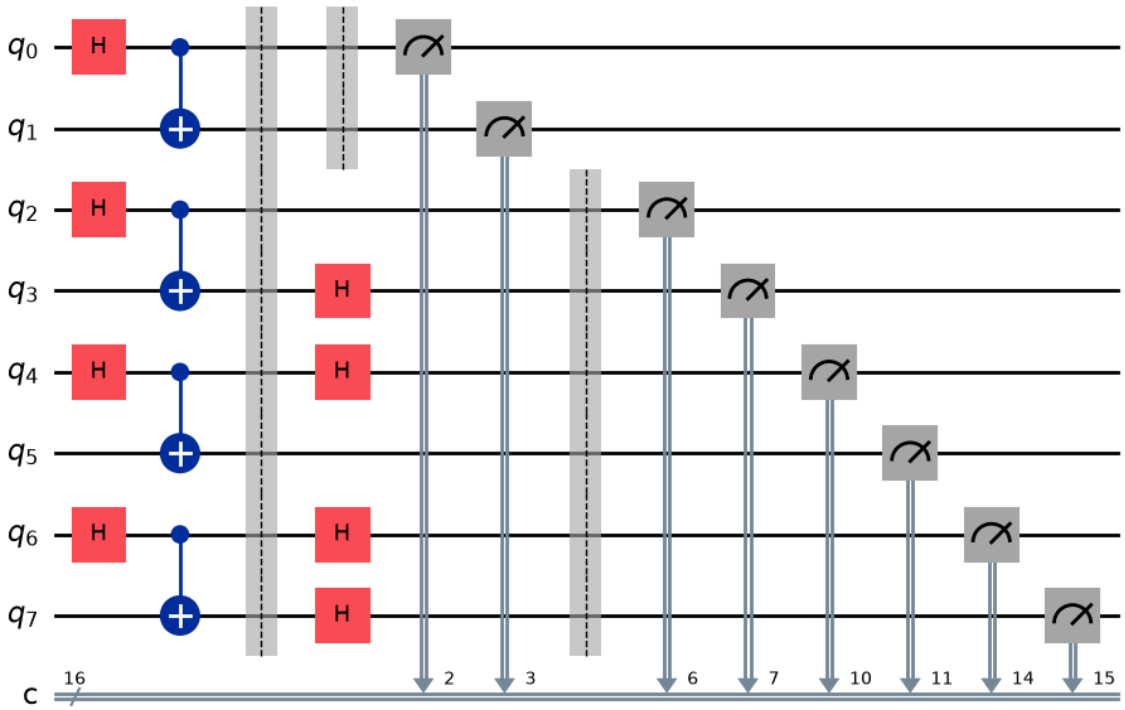
# Build the individual circuits first in a list
combined_subcircuits = []
for j, view in enumerate(views):
    circuit = MeasureWithView(
        BellCircuit('q'+view, 'c'+view+'0'),
        view
    )
    combined_subcircuits.append(circuit)

# Create an 8-qubit and 16-clbit container (4 clbits per sub-circuit)
circ_combined = QuantumCircuit(8, 16)

for j, circuit in enumerate(combined_subcircuits):
    circ_combined.compose(
        circuit,
        qubits=range(2*j, 2*(j+1)),
        clbits=range(4*j, 4*(j+1)),
        inplace=True
    )

# Draw the final combined circuit
circ_combined.draw('mpl')
```

Out[58]:



<h3 id="Parallelizing-Copies-of-Experiments-(7%25)">Parallelizing Copies of Experiments (7%)</h3> Now we have a circuit that will execute all 4 experiments side by side, but this uses only 8 qubits. We can still utilize more of the hardware resources by creating copies of this circuit across the entire qubit topology.

 Fill in the rest of the code to create one wide, shallow circuit containing several copies of each measurement view experiment. Be sure to provide distict names for the classical and quantum registers in each experiment circuit. This will make it easier to process later. You can refer to the previous code cell for guidance.

In [60]:

```
from qiskit import QuantumCircuit

num_qubits = 8 * (int(0.8 * backend.num_qubits) // 8)

# Initialize circ_real outside the loops
circ_real = QuantumCircuit(num_qubits, 2 * num_qubits)

for i in range(0, num_qubits, 8):
    for j, view in enumerate(views):
        # Create the individual CHSH experiment circuit with distinct register name
        circuit = MeasureWithView(
            BellCircuit(f'q_{i}_{j}', f'c_{i}_{j}_0'),
            view
        )

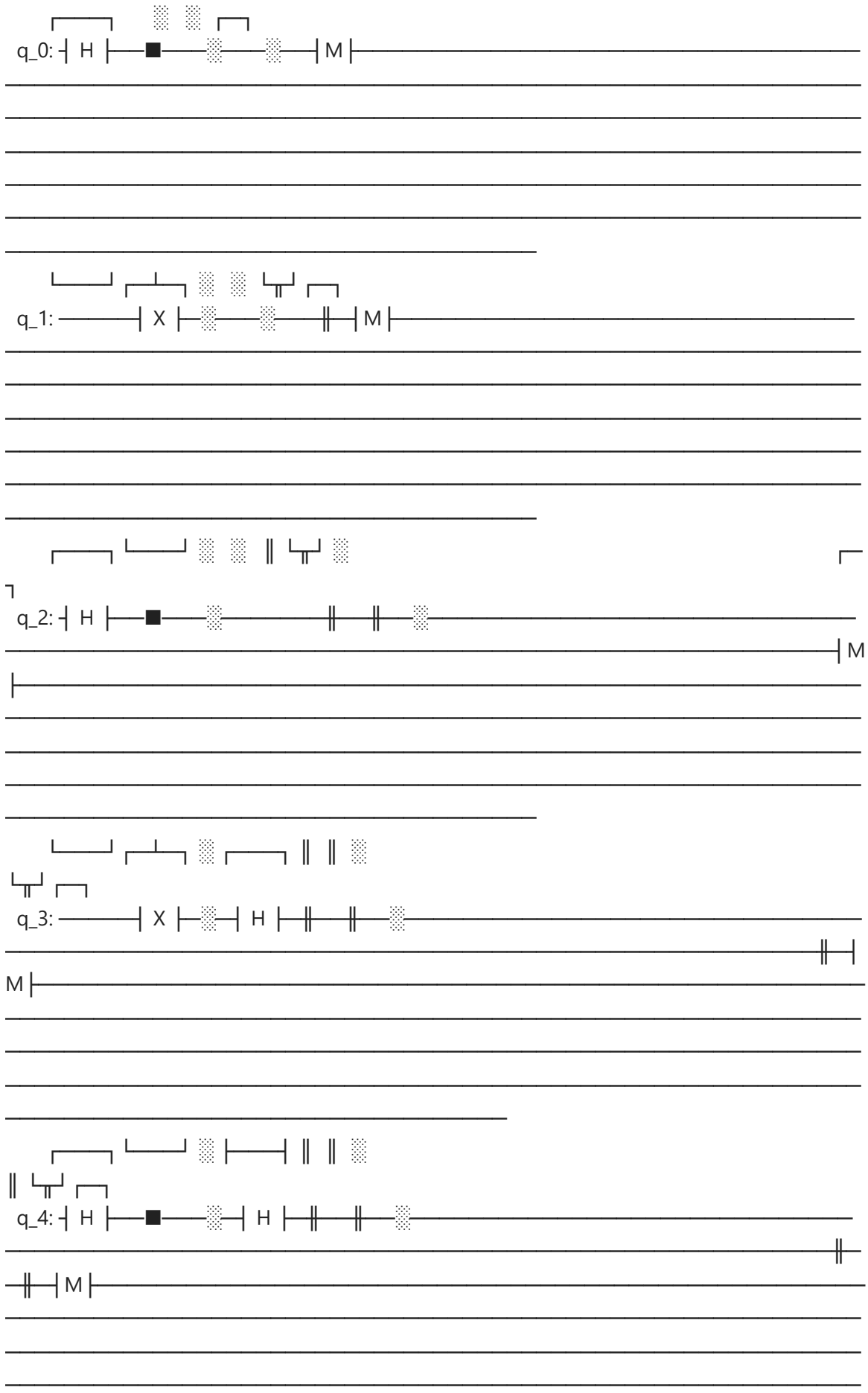
        # Compose onto the specific global qubit and classical bit ranges
        q_start = i + 2 * j
        c_start = 2 * (i + 2 * j)

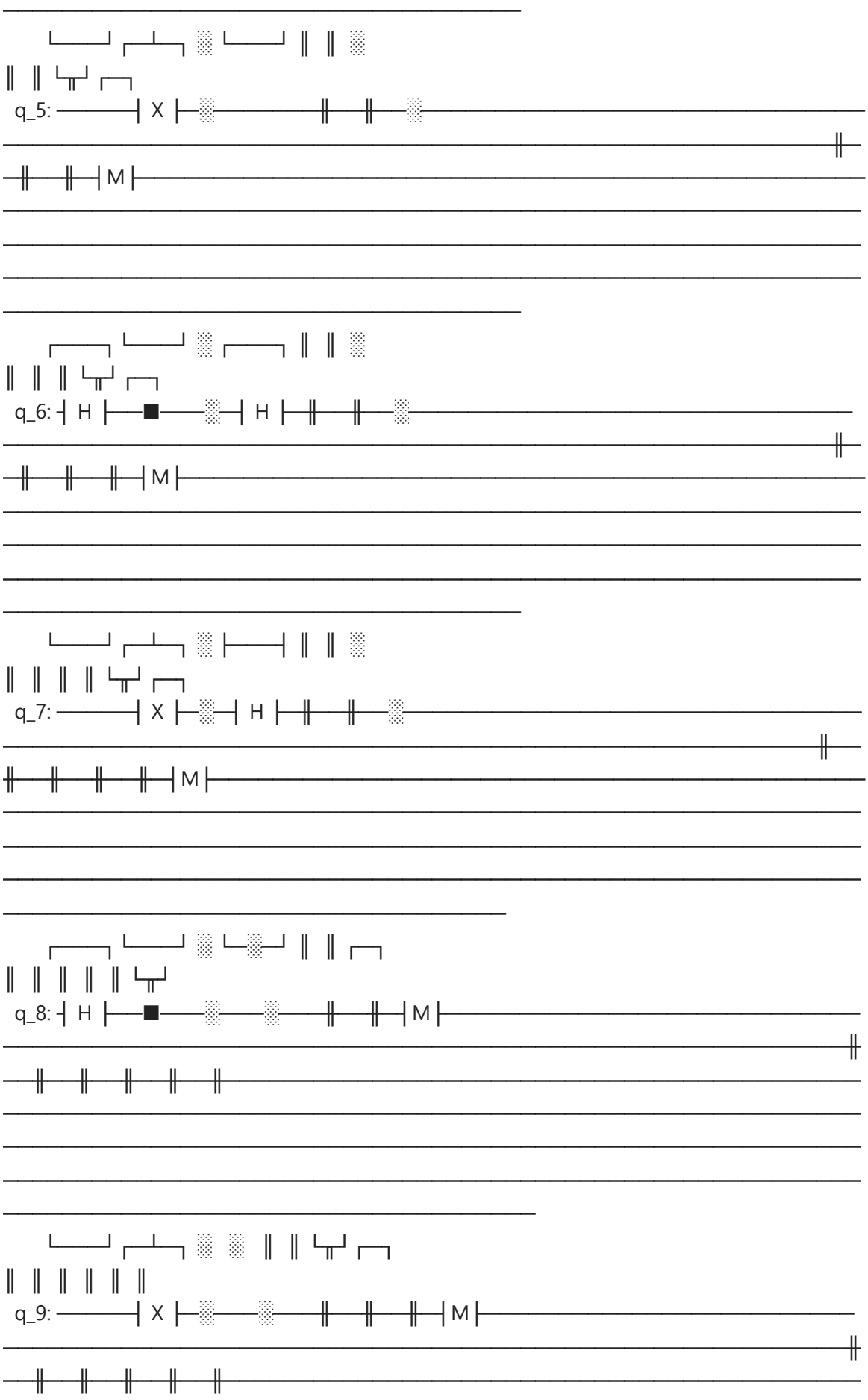
        circ_real.compose(
            circuit,
```

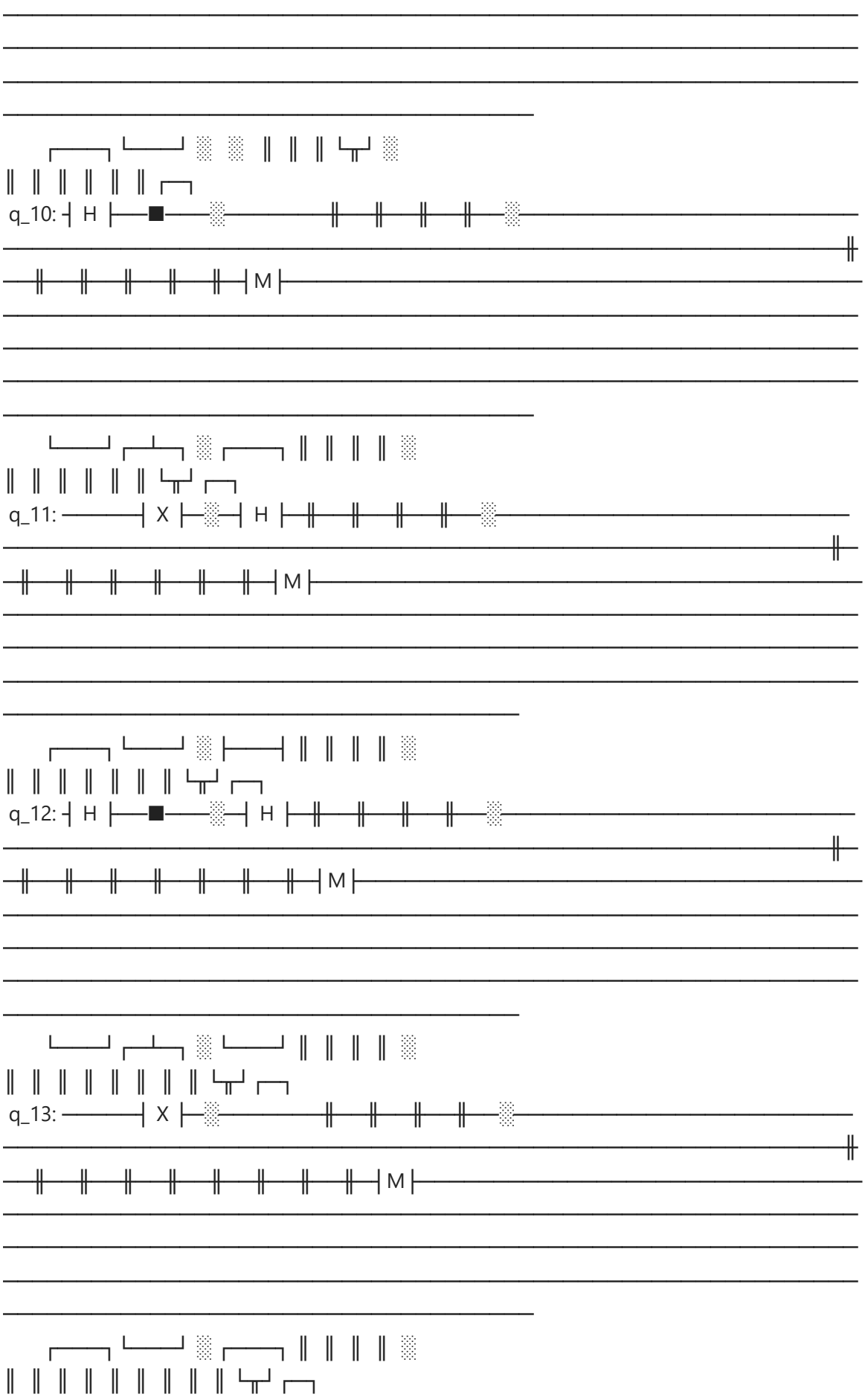
```
            qubits=range(q_start, q_start + 2),
            clbits=range(c_start, c_start + 4),
            inplace=True
        )

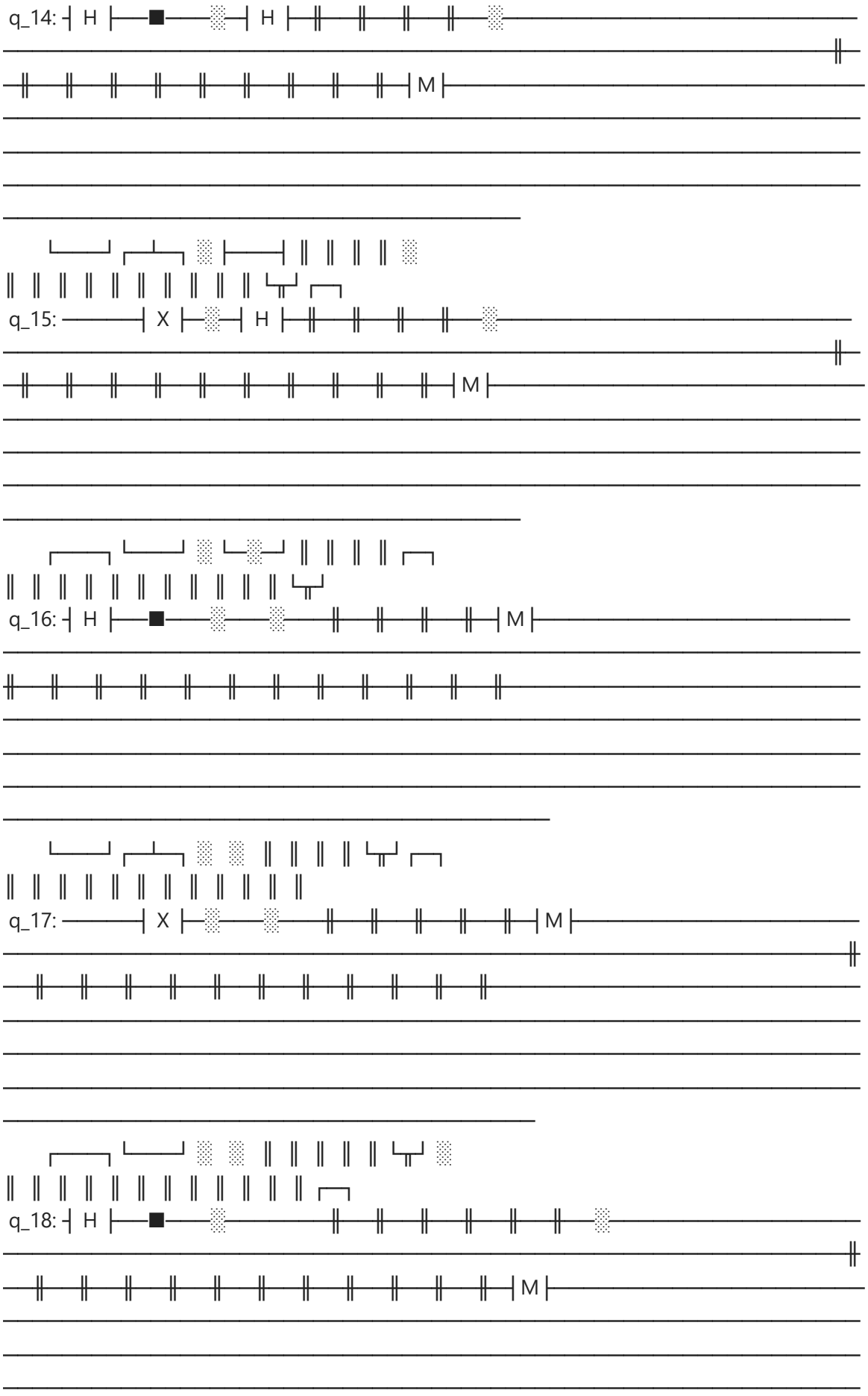
# Draw the final scaled circuit
circ_real.draw(fold=-1)
```

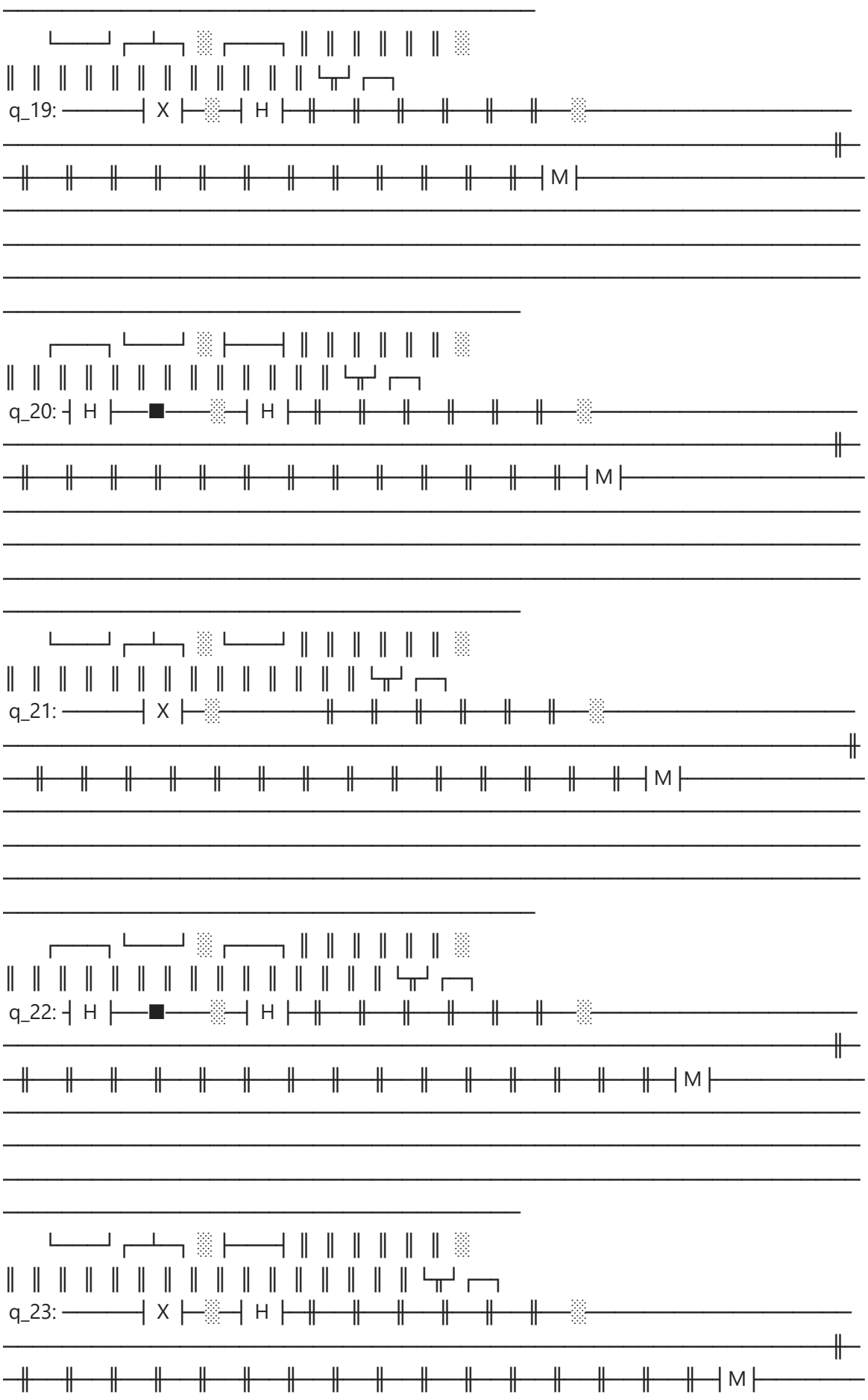
Out[60]:

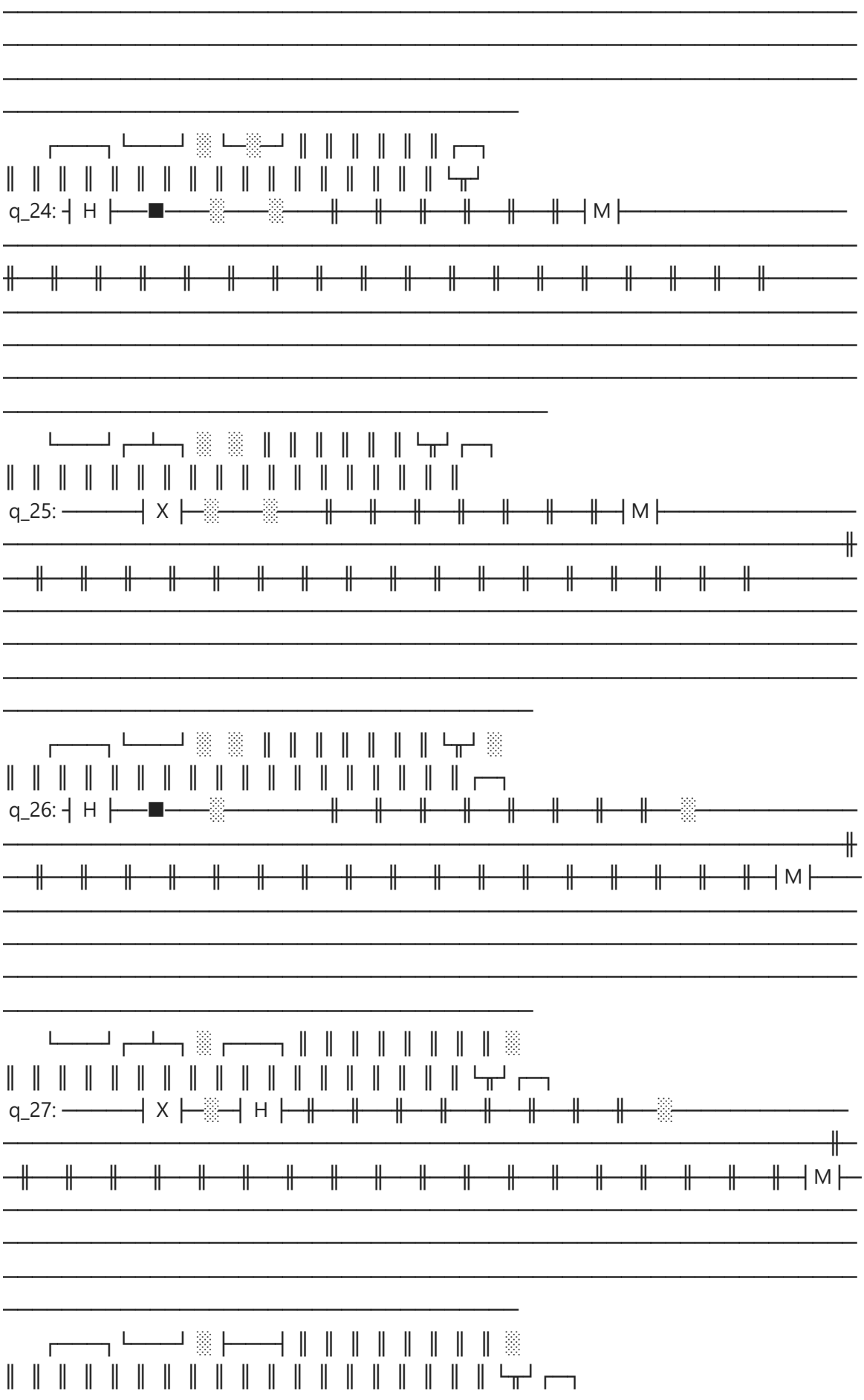


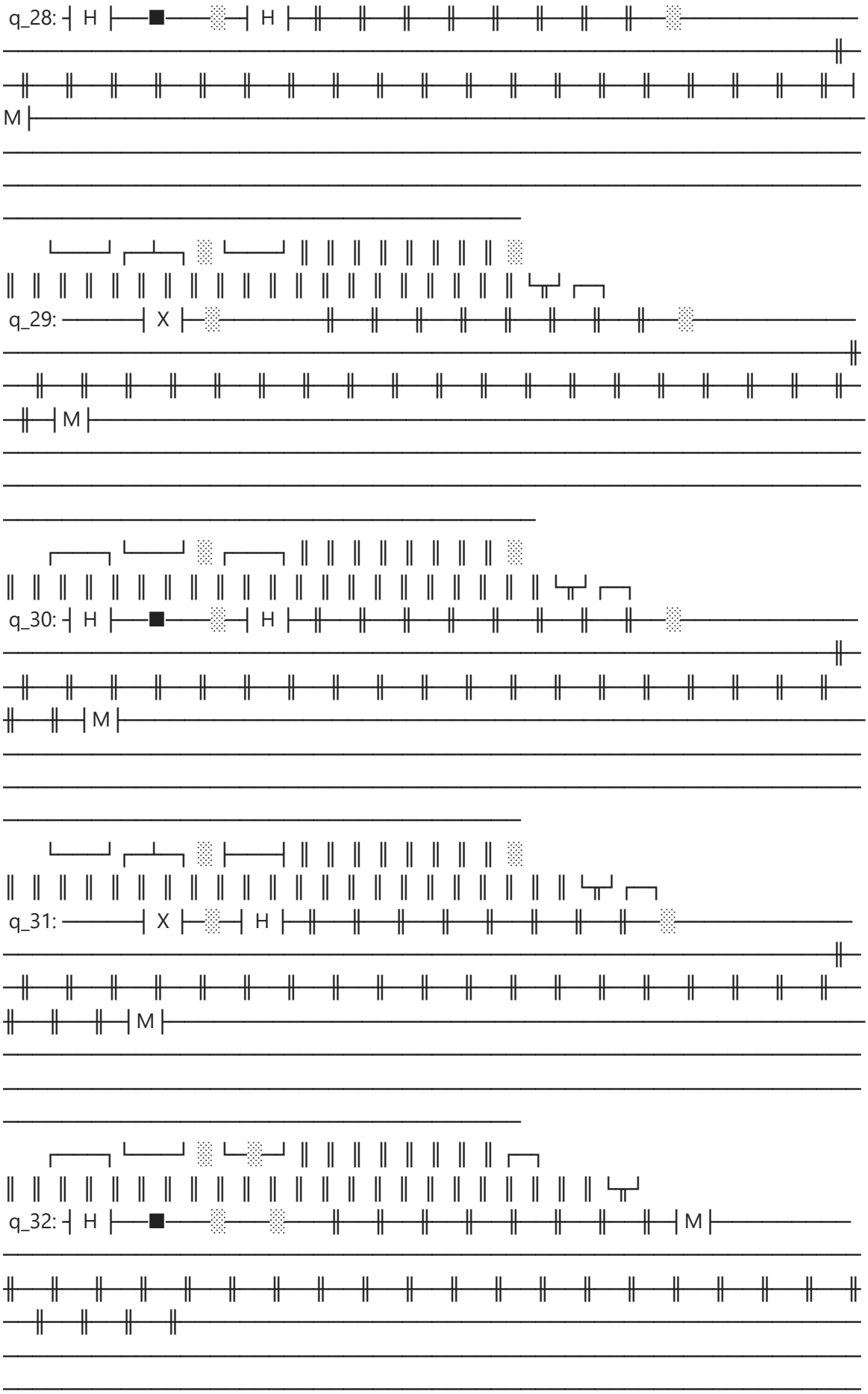

q_5:
X
M
q_6:
H
H
M
q_7:
X
H
M
q_8:
H
M
q_9:
X
M

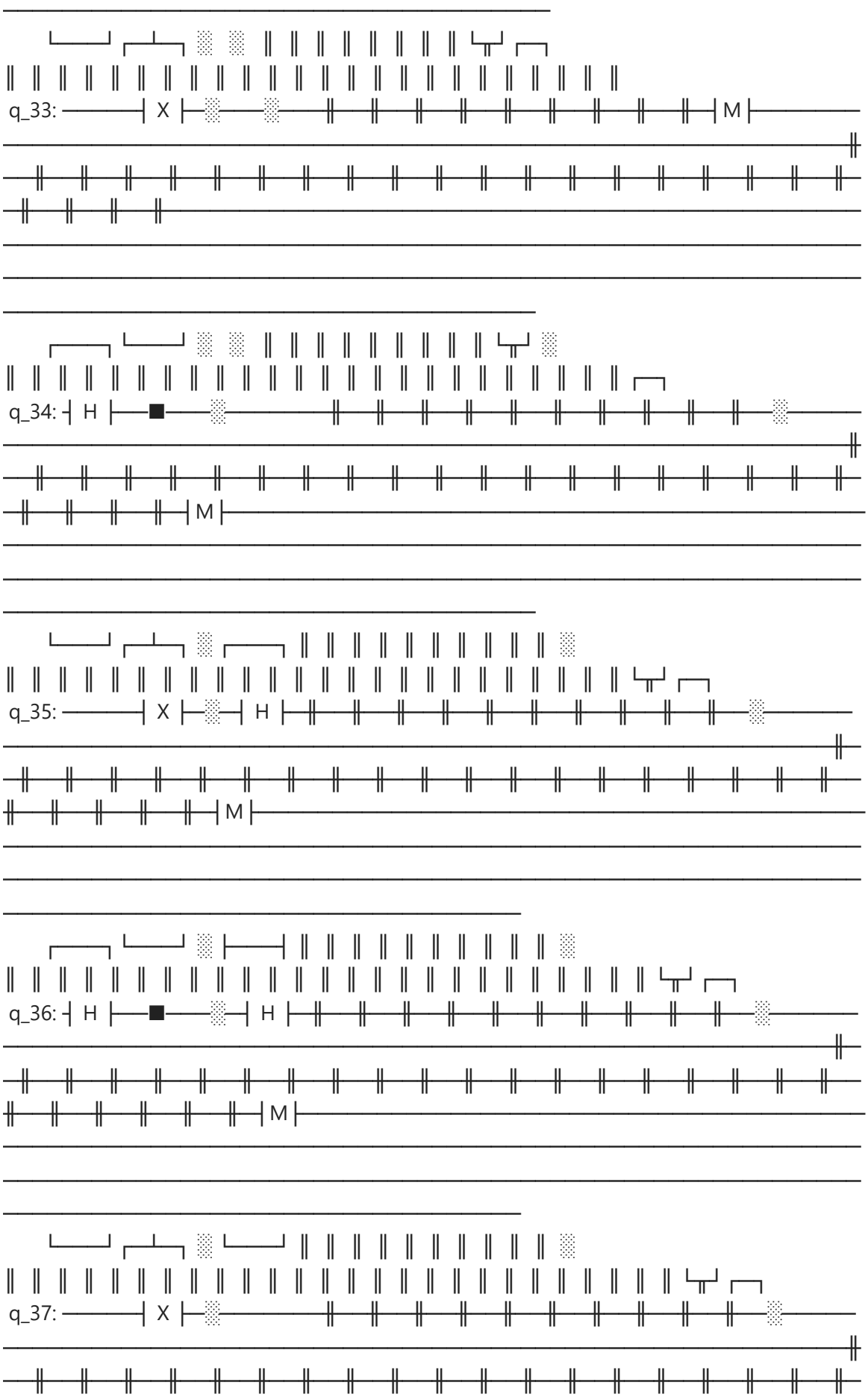

q_10:
H
M
q_11:
X
H
M
q_12:
H
H
M
q_13:
X
M

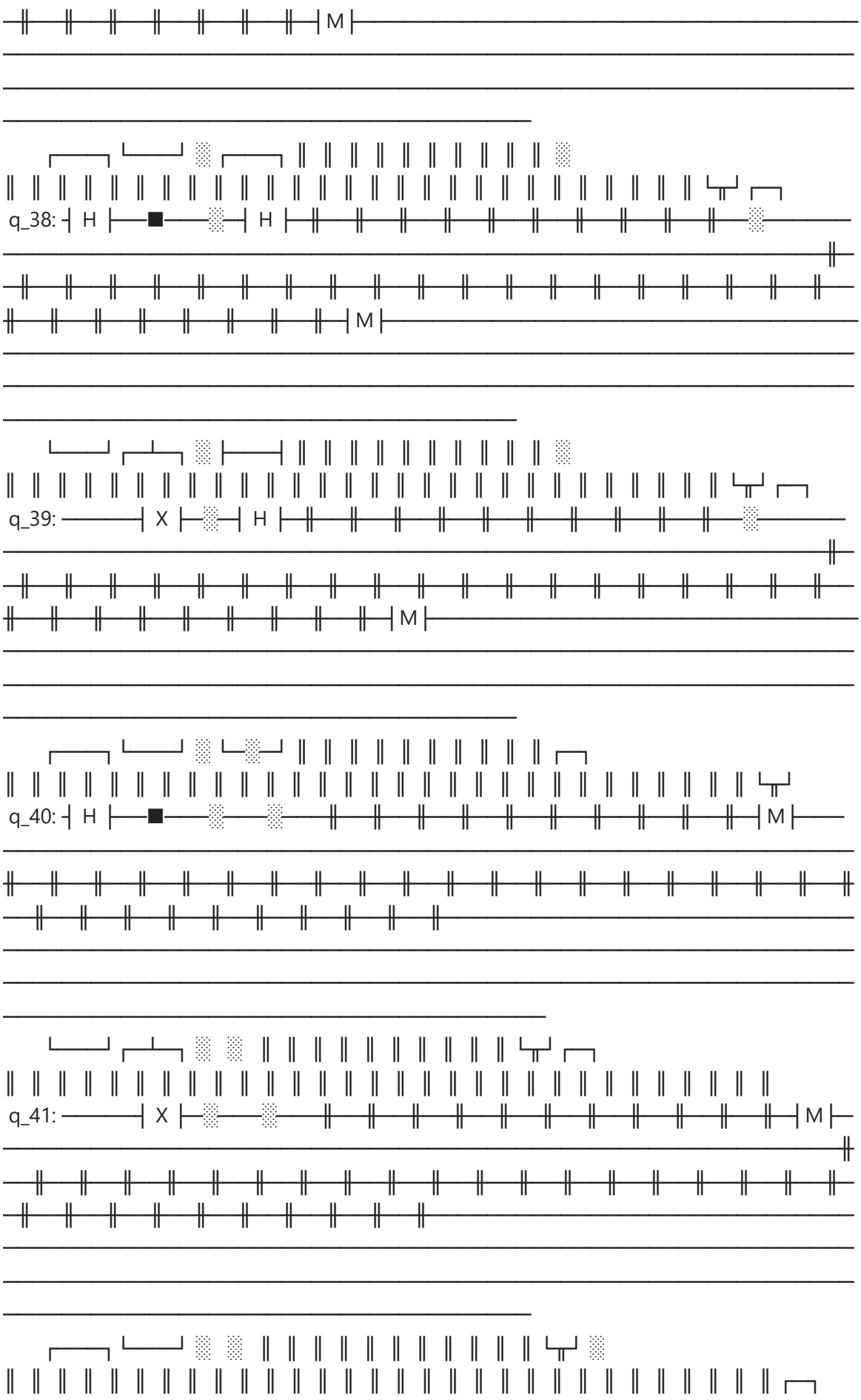

q_14:
H
H
M
q_15:
X
H
M
q_16:
H
M
q_17:
X
M
q_18:
H
M

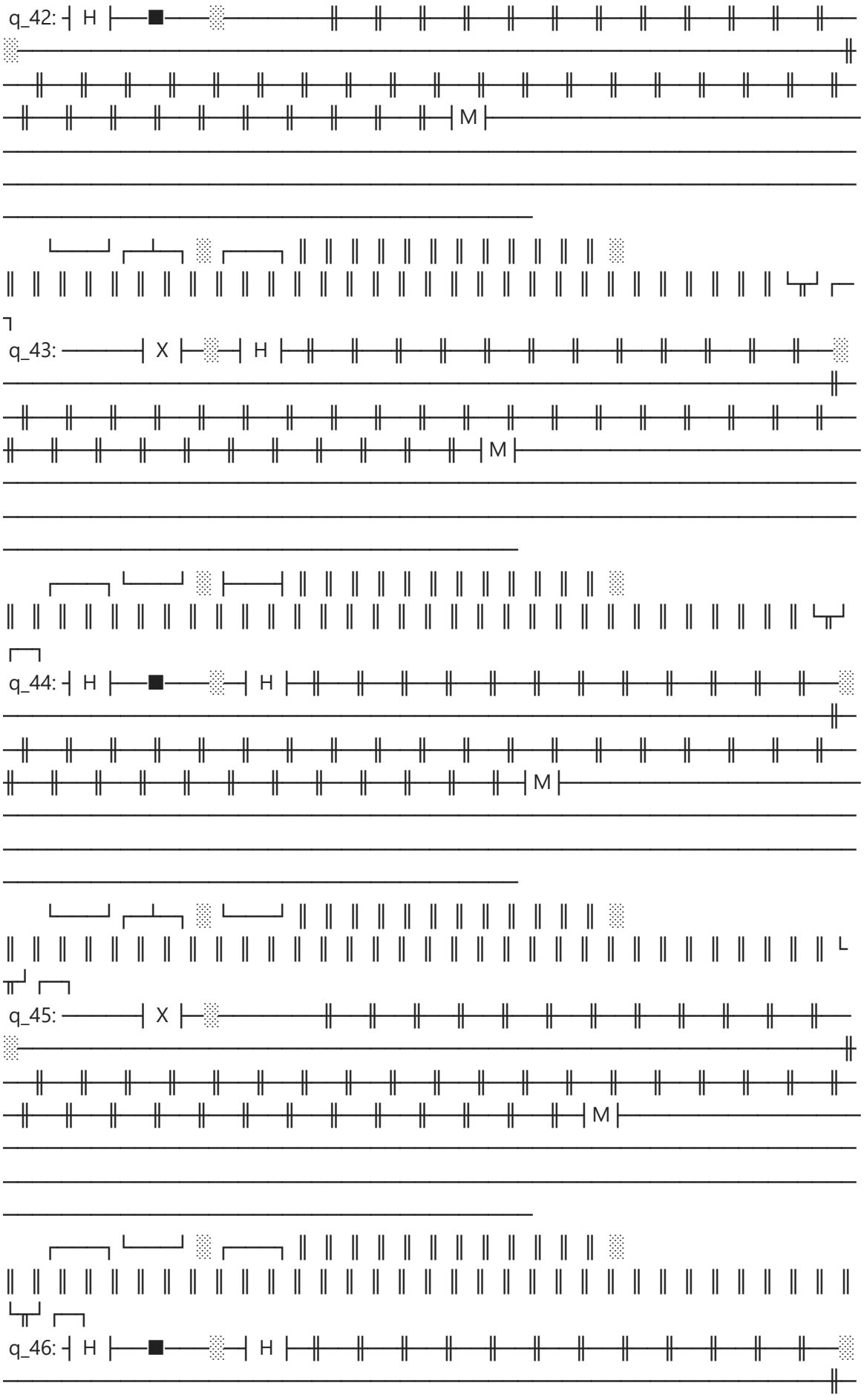

q_19:
X
H
M
q_20:
H
H
M
q_21:
X
M
q_22:
H
H
M
q_23:
X
H
M

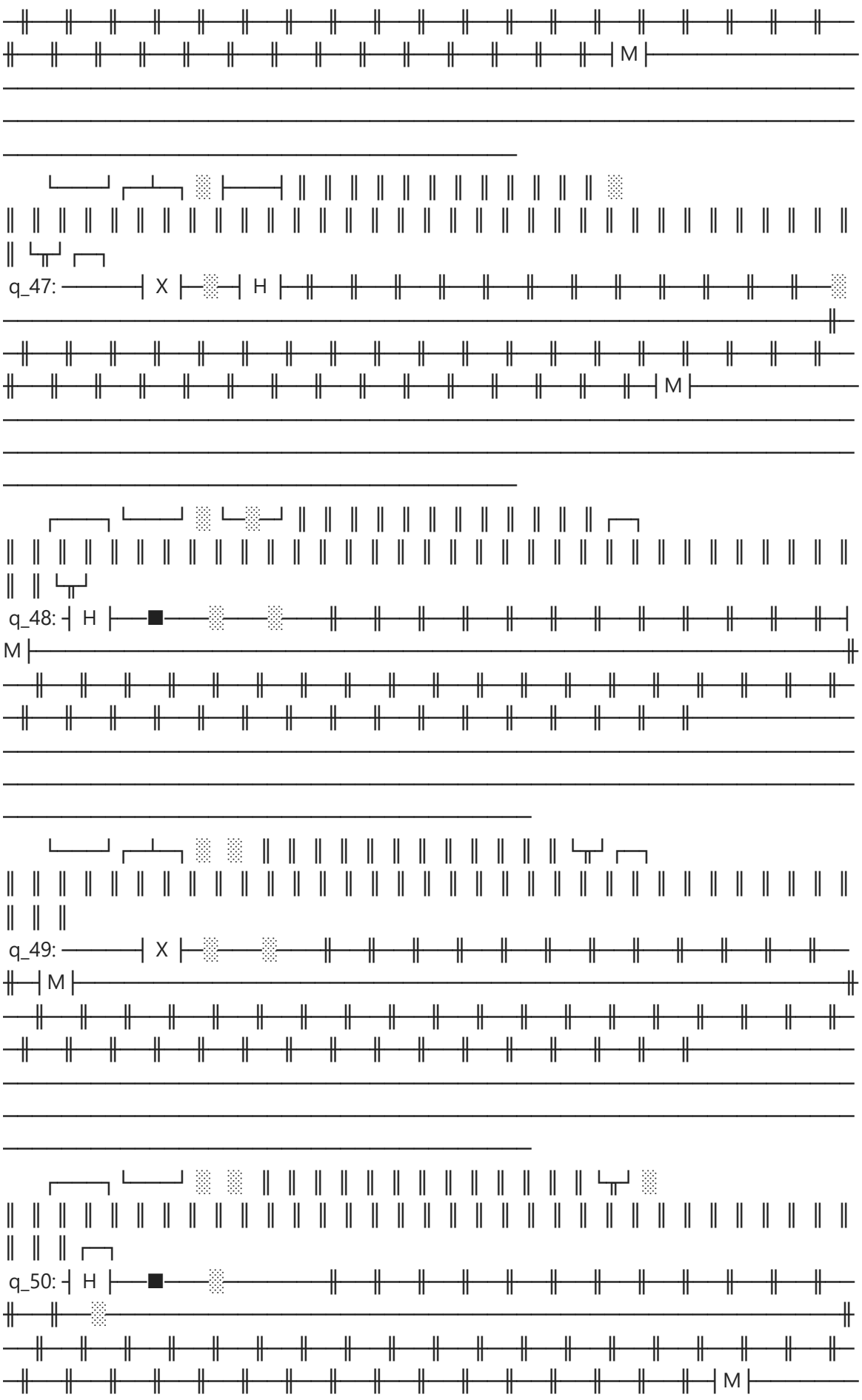

q_24:
H
M
q_25:
X
M
q_26:
H
M
q_27:
X
H
M

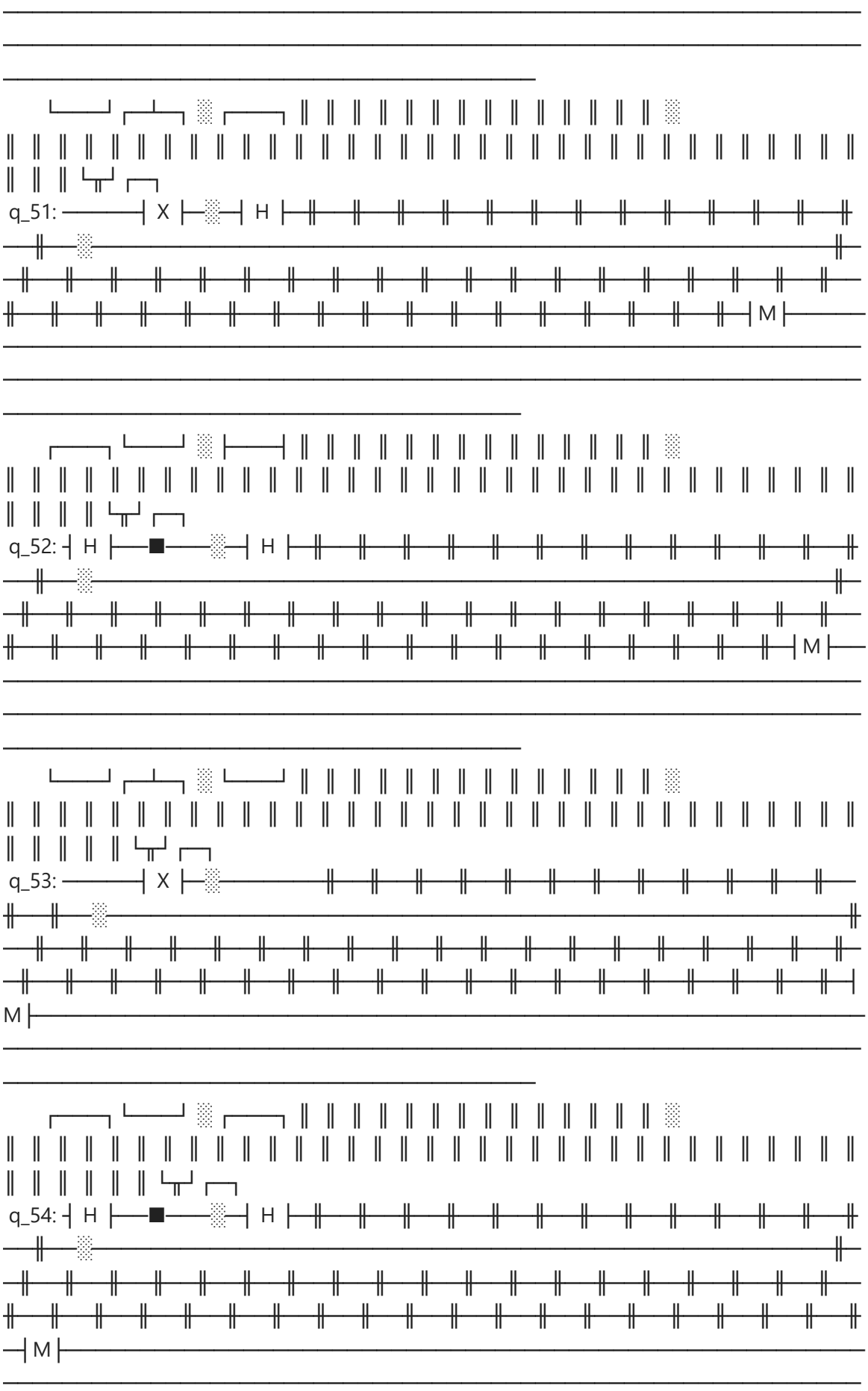

q_28:
H
H
M
q_29:
X
M
q_30:
H
H
M
q_31:
X
H
M
q_32:
H
M

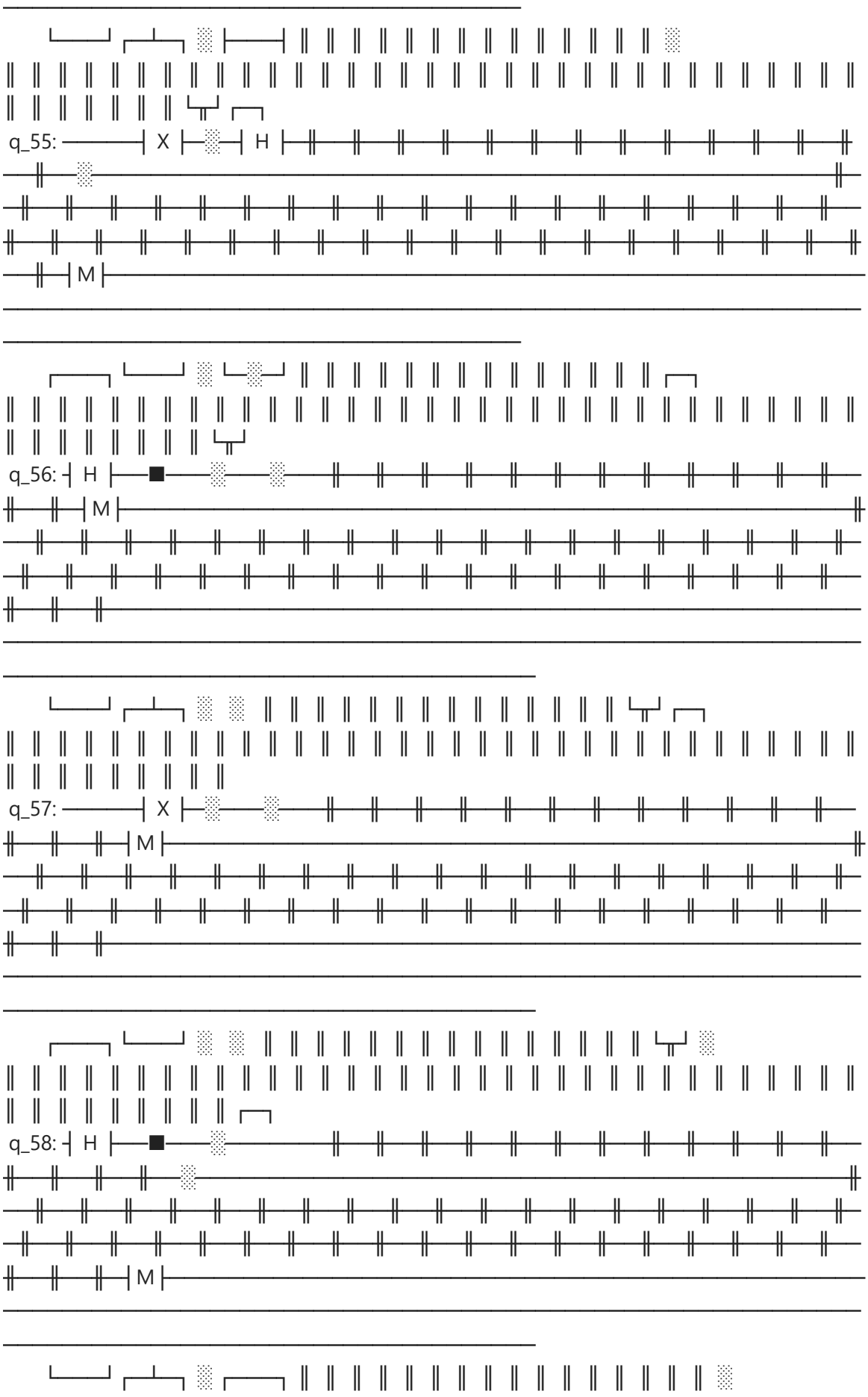

q_33:
X
M
q_34:
H
M
q_35:
X
H
M
q_36:
H
H
M
q_37:
X

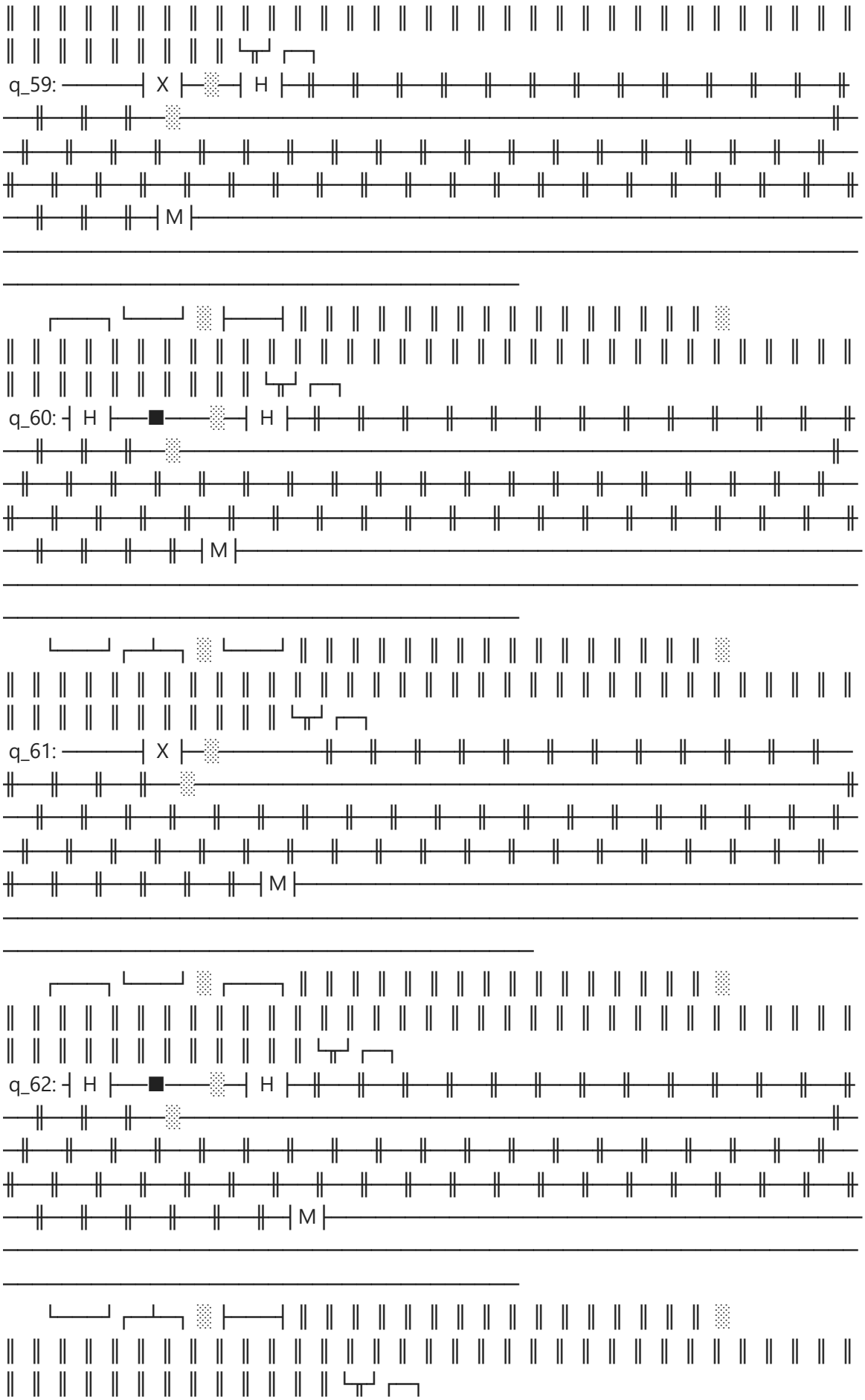

M
q_38:
H
H
M
q_39:
X
H
M
q_40:
H
M
q_41:
X
M

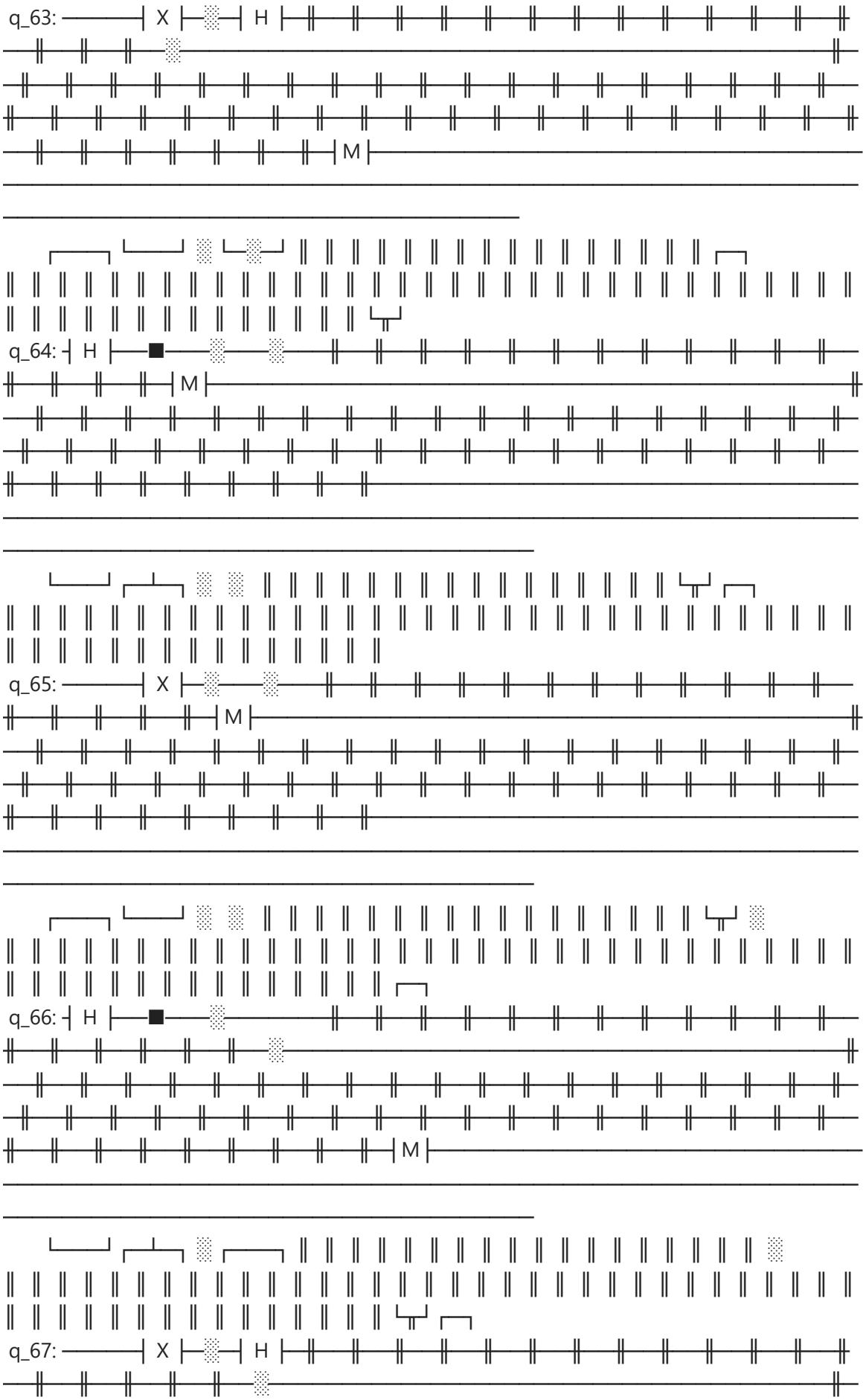

q_42:
H
M
q_43:
X
H
M
q_44:
H
H
M
q_45:
X
M
q_46:
H
H

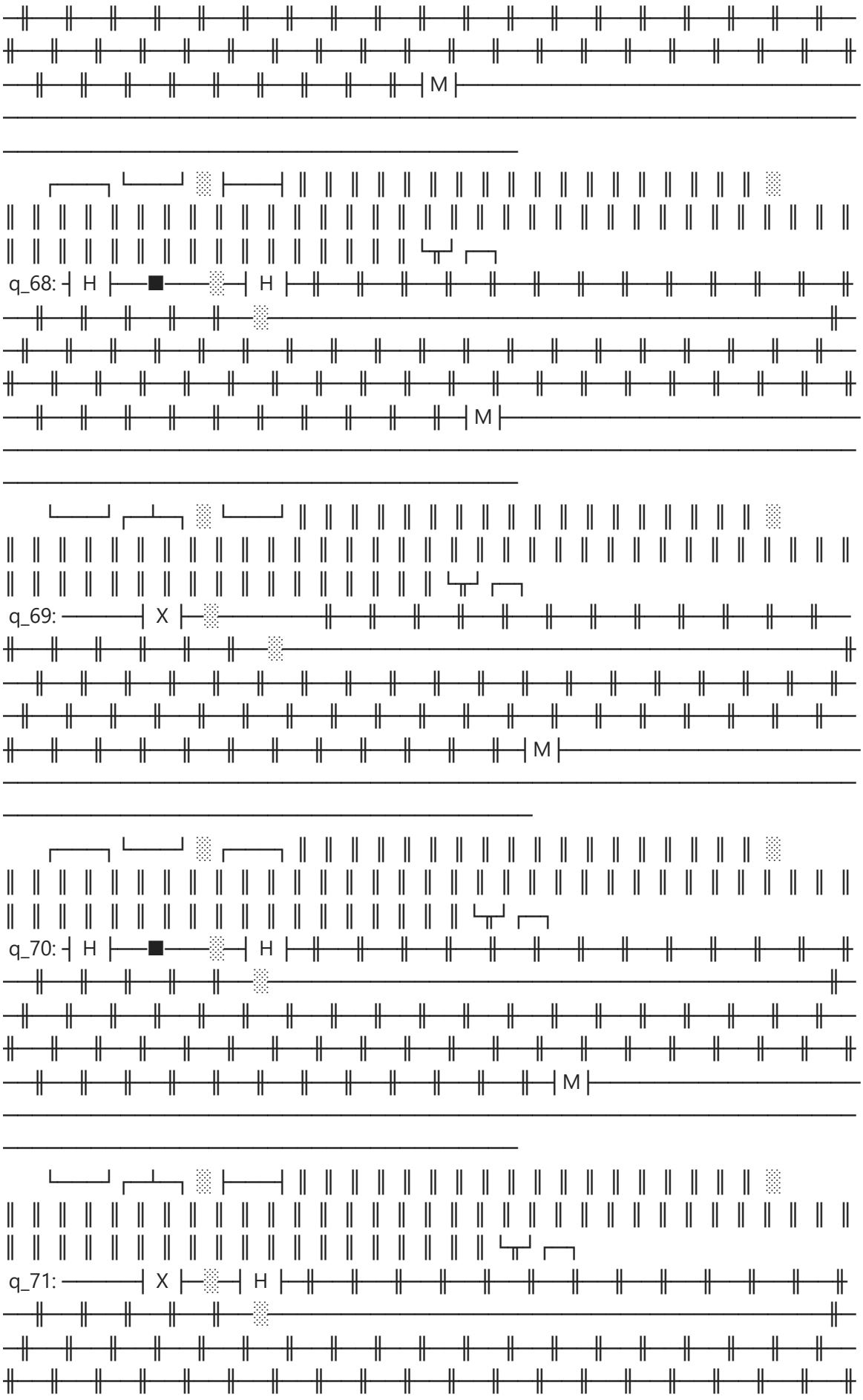

M
q_47:
X
H
M
q_48:
H
M
q_49:
X
M
q_50:
H
M

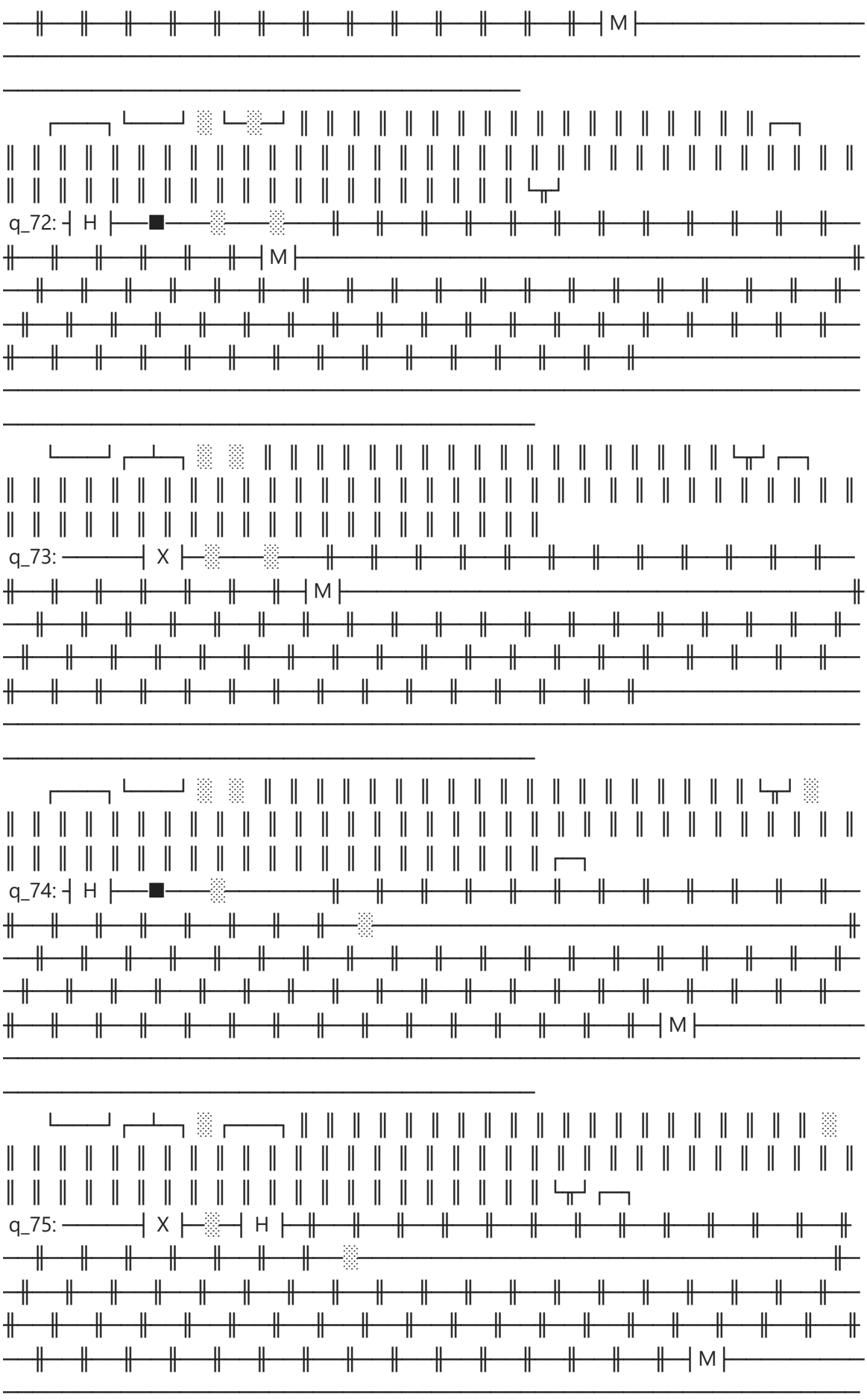

q_51:
X
H
M
q_52:
H
H
M
q_53:
X
M
q_54:
H
H
M

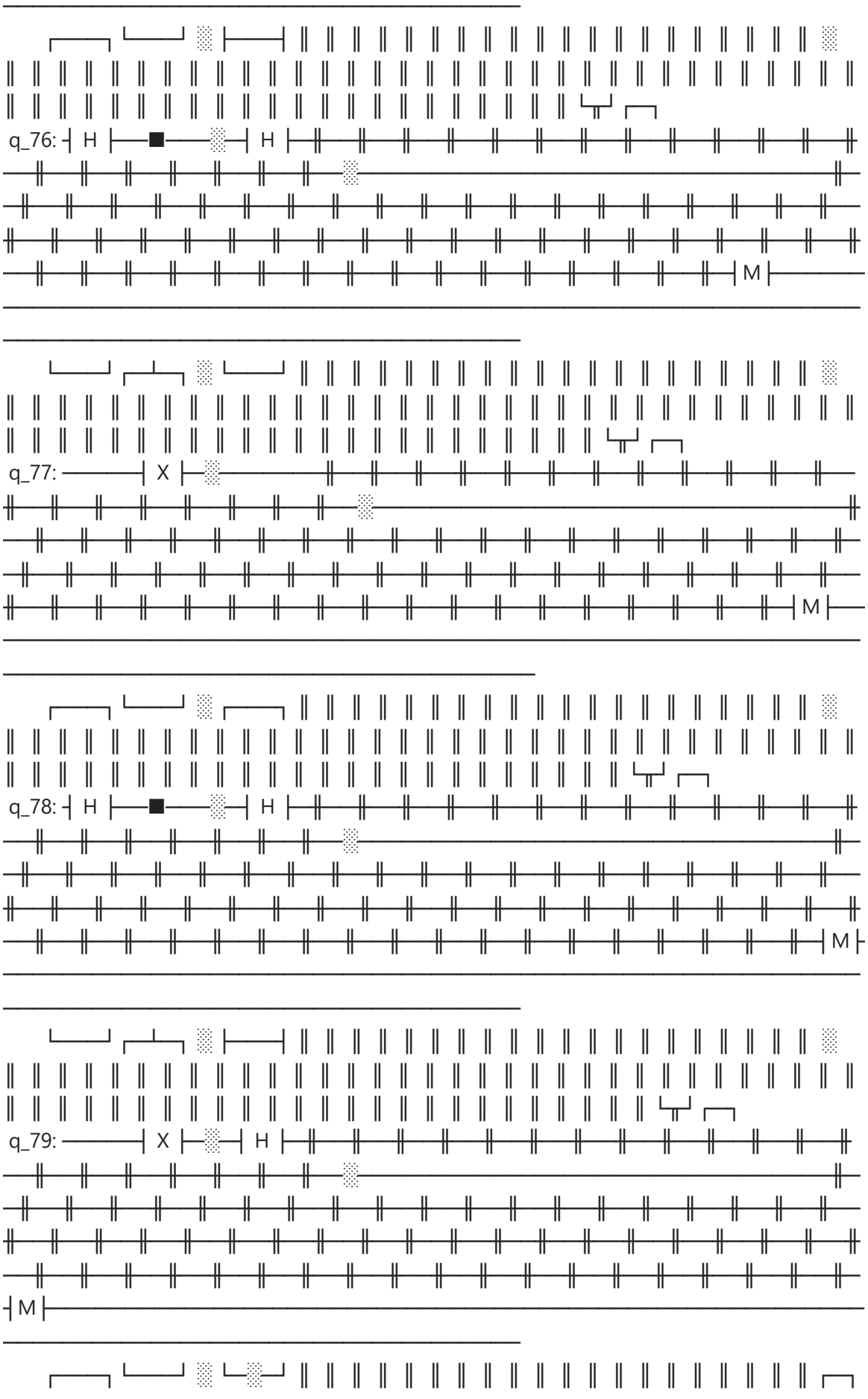

q_55:
X
H
M
q_56:
H
M
q_57:
X
M
q_58:
H
M

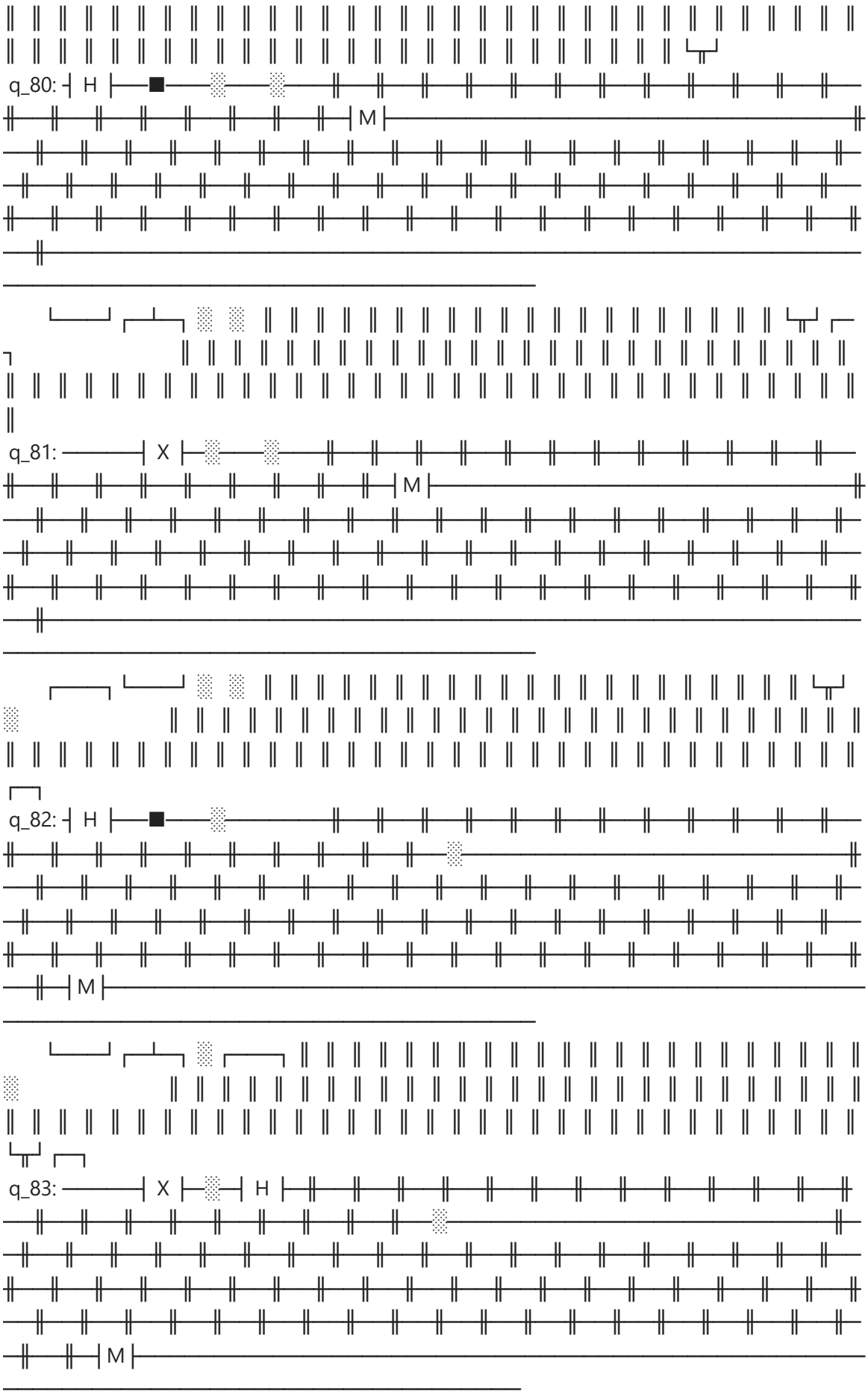

q_59:
X
H
M
q_60:
H
H
M
q_61:
X
M
q_62:
H
H
M

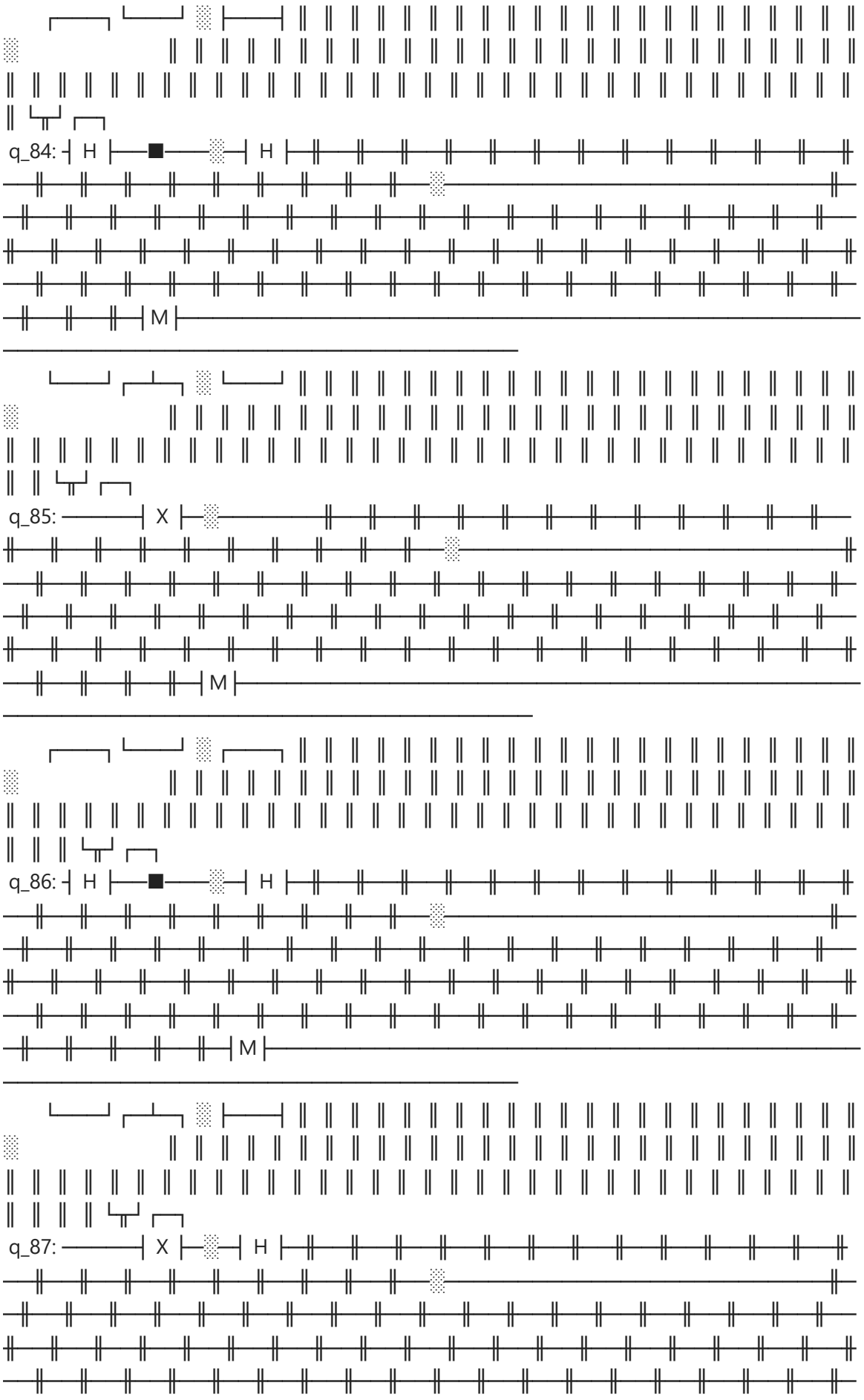

q_63:
X
H
M
q_64:
H
M
q_65:
X
M
q_66:
H
M
q_67:
X
H

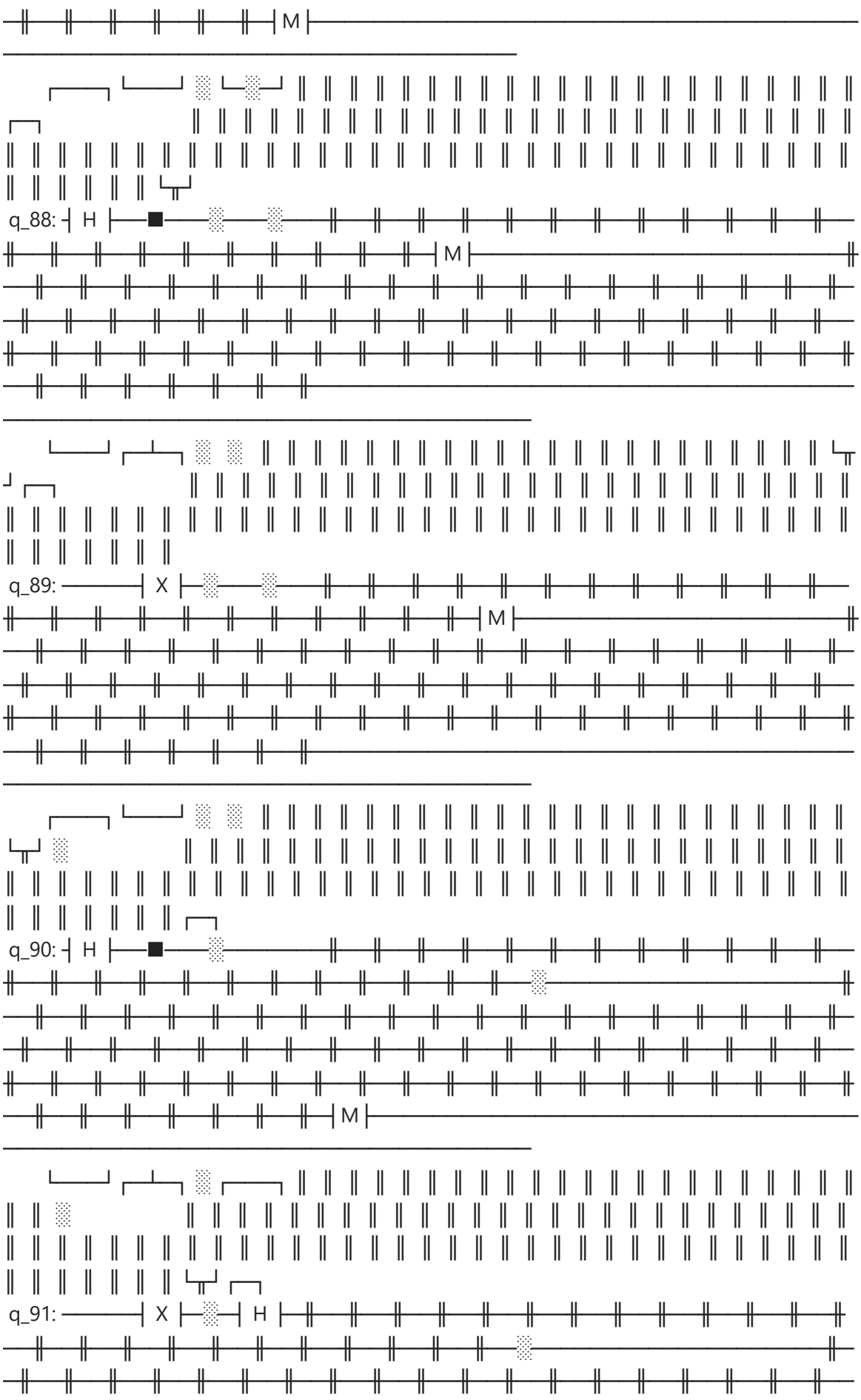

M
q_68:
H
H
M
q_69:
X
M
q_70:
H
H
M
q_71:
X
H

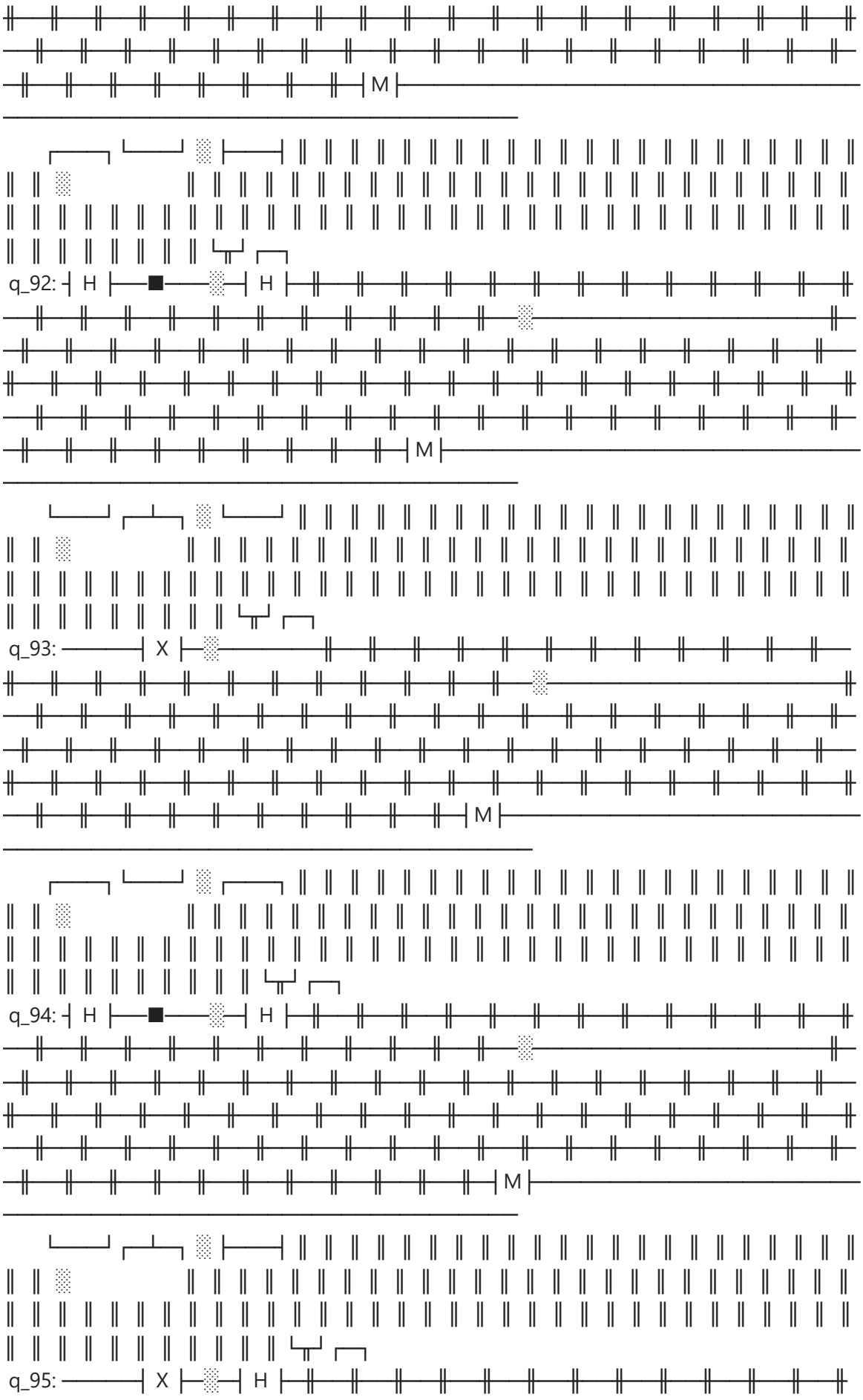

M
q_72:
H
M
q_73:
X
M
q_74:
H
M
q_75:
X
H
M

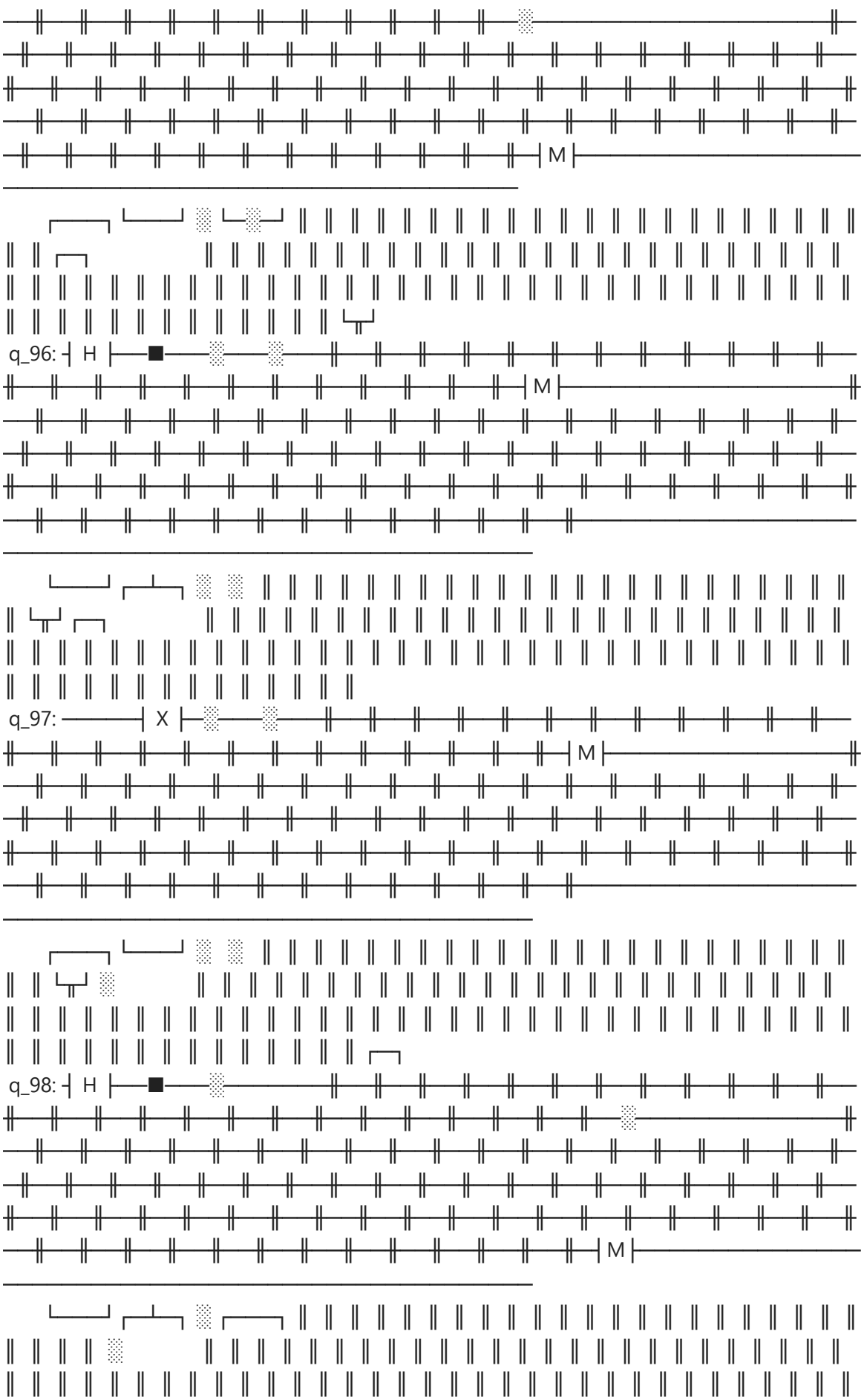

q_76:
H
H
M
q_77:
X
M
q_78:
H
H
M
q_79:
X
H
M

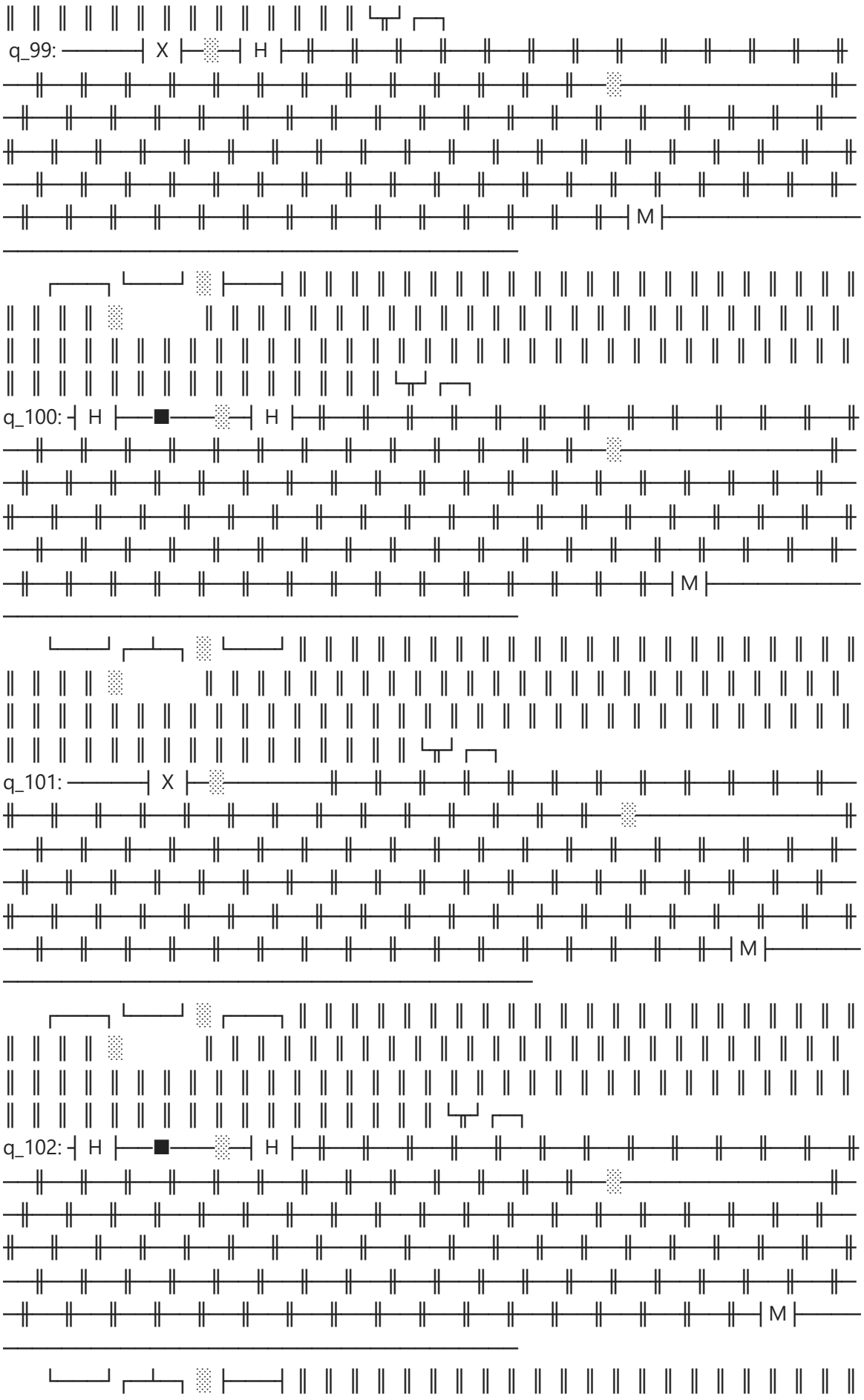

q_80:
H
M
q_81:
X
M
q_82:
H
M
q_83:
X
H
M

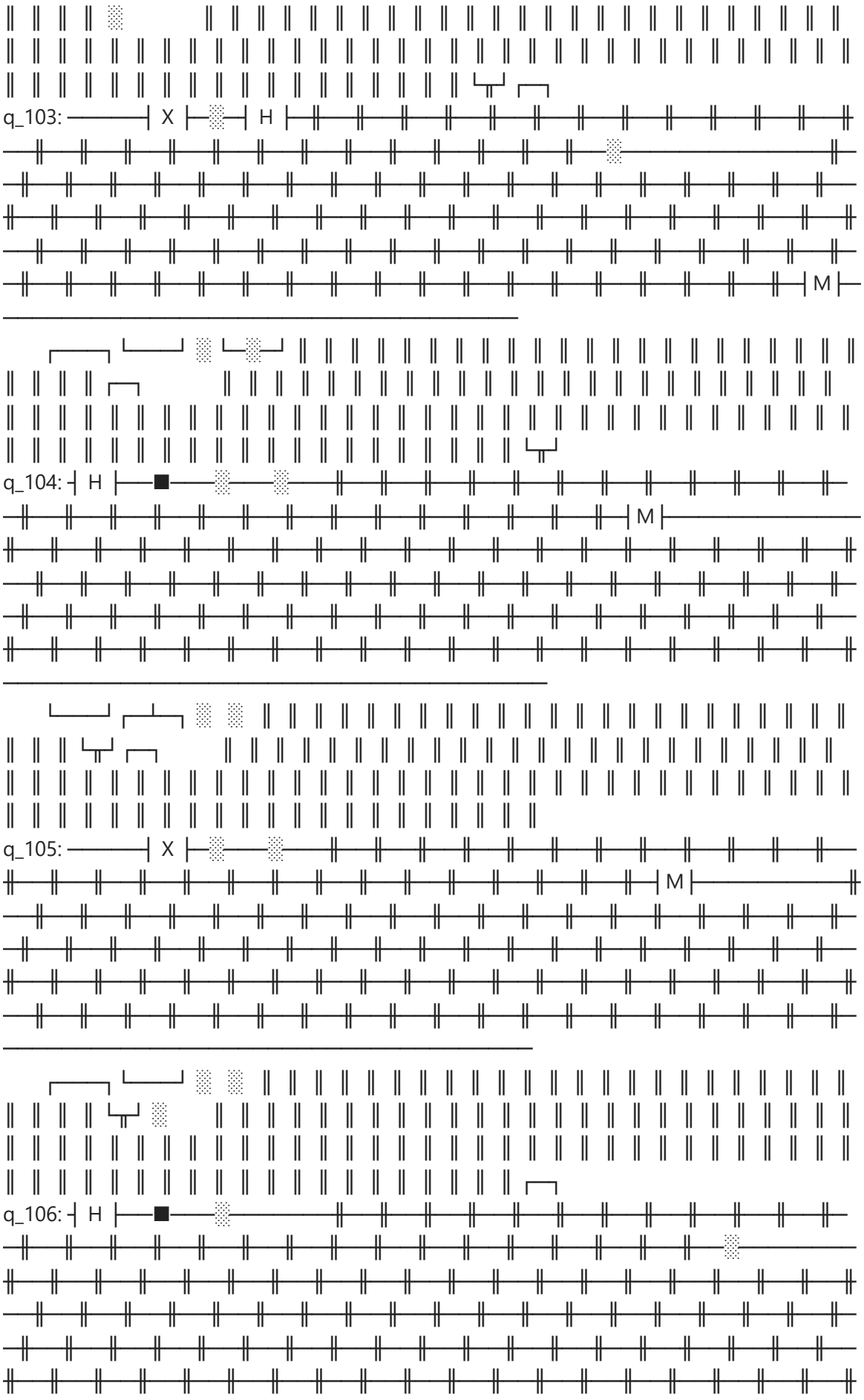

q_84:
H
H
M
q_85:
X
M
q_86:
H
H
M
q_87:
X
H

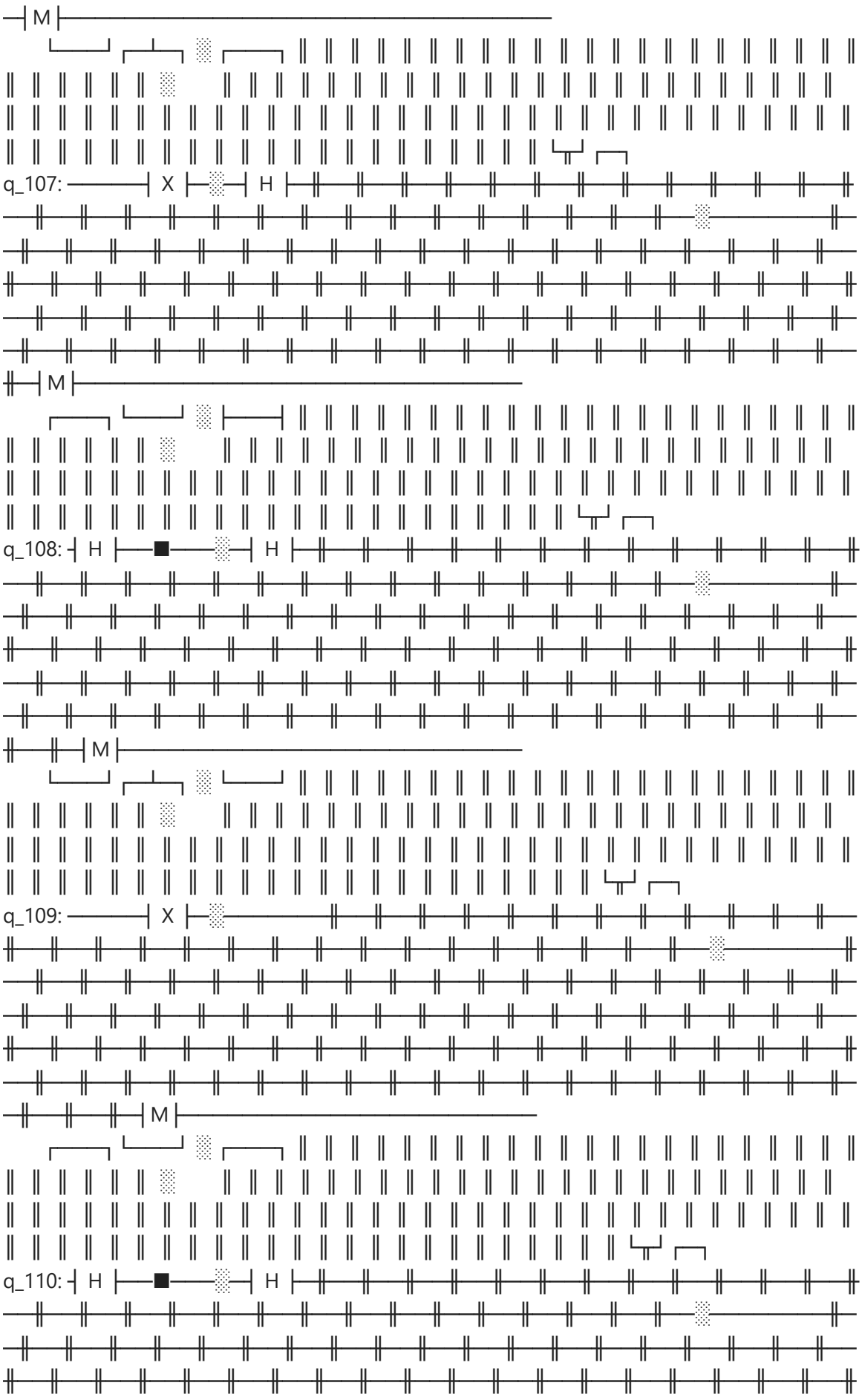

M
q_88:
H
M
q_89:
X
M
q_90:
H
M
q_91:
X
H

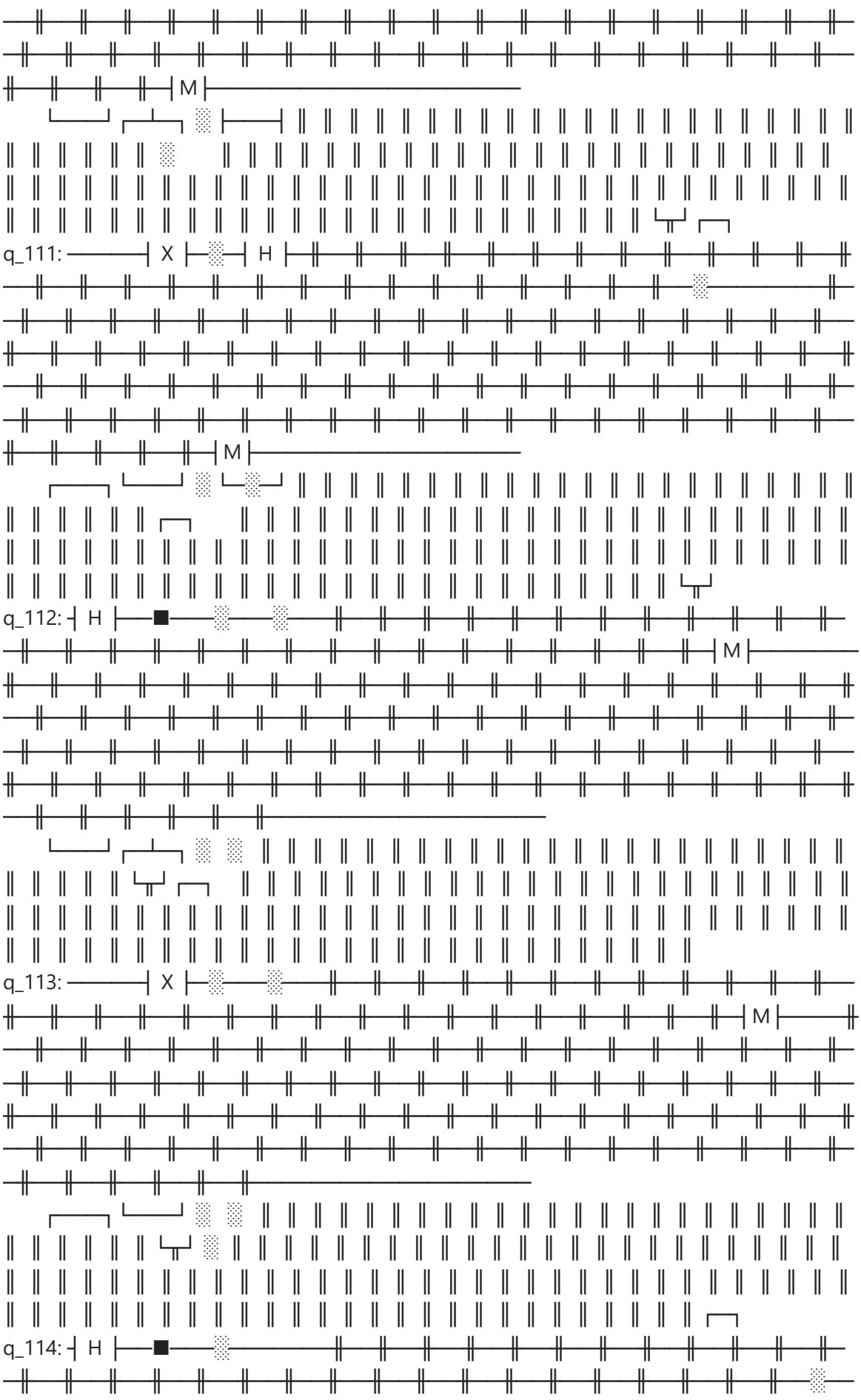

M
q_92:
H
H
M
q_93:
X
M
q_94:
H
H
M
q_95:
X
H

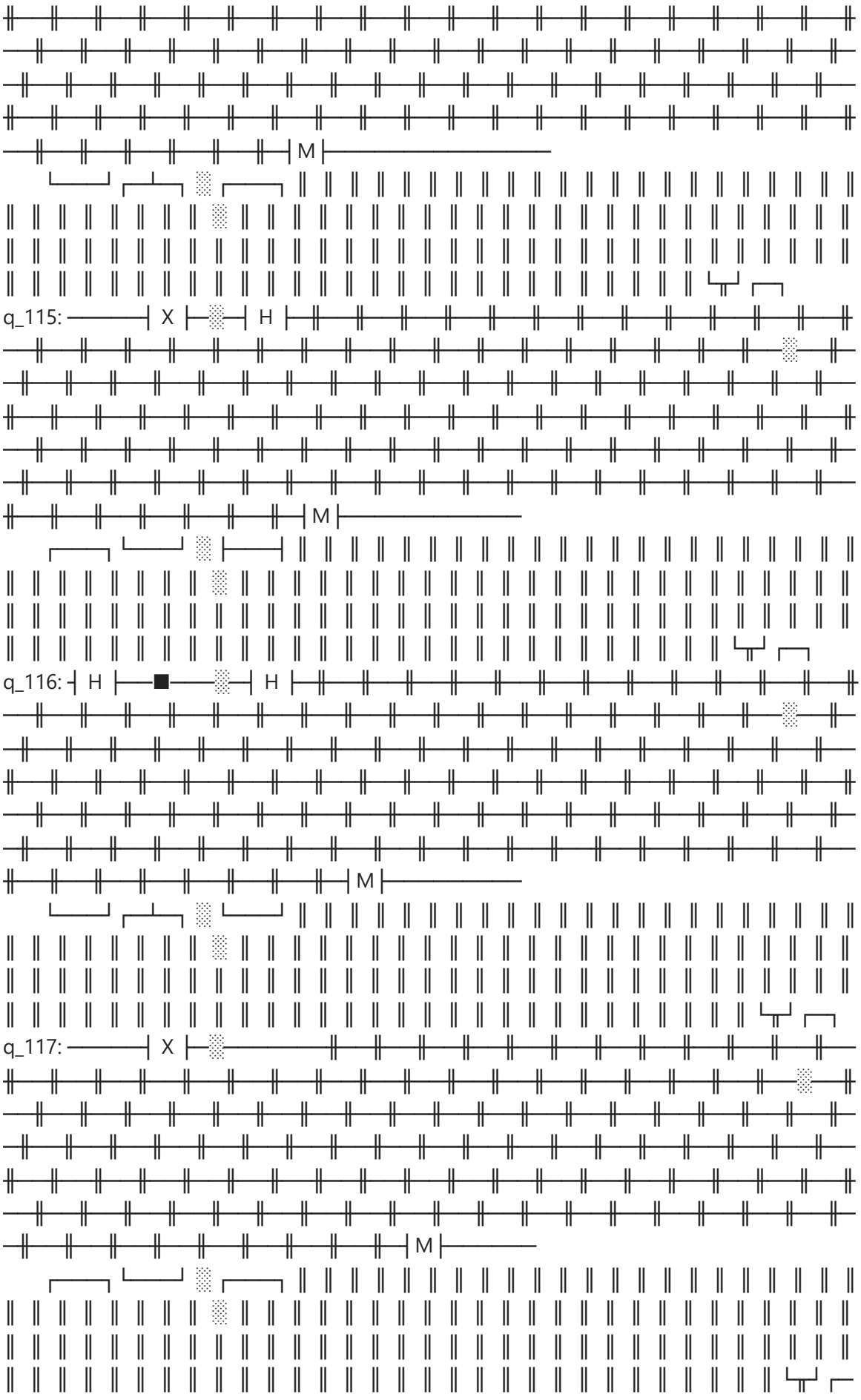

M
q_96:
H
M
q_97:
X
M
q_98:
H
M

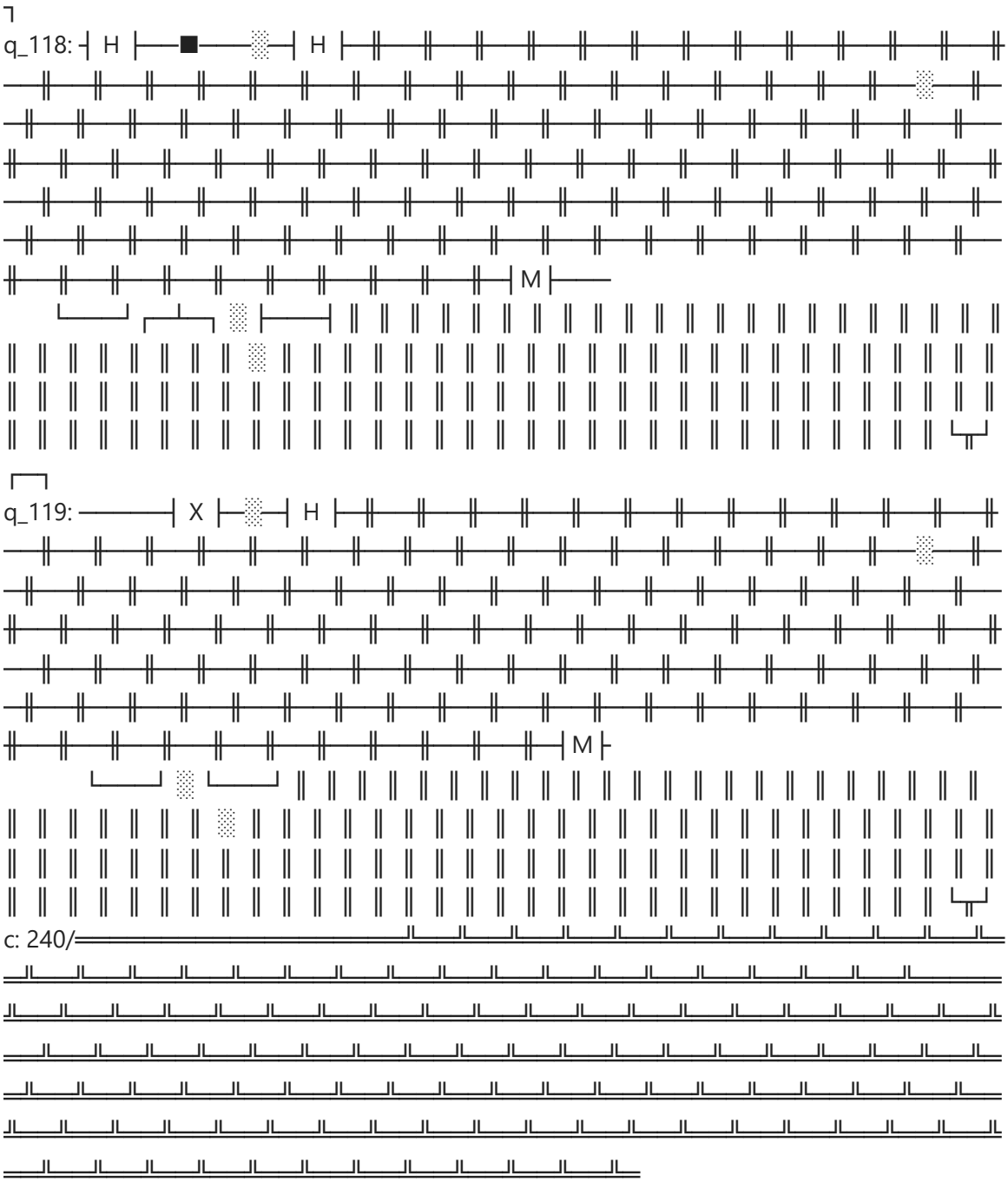

q_99:
X
H
M
q_100:
H
H
M
q_101:
X
M
q_102:
H
H
M


q_103:
X
H
M
q_104:
H
M
q_105:
X
M
q_106:
H


M
q_107:
X
H
M
q_108:
H
H
M
q_109:
X
M
q_110:
H
H


M
q_111:
X
H
M
q_112:
H
M
q_113:
X
M
q_114:
H


M
q_115:
X
H
M
q_116:
H
H
M
q_117:
X
M



2 3 18 19 34 35 50 51 66 67 82 83 98 99114115130131146147162163178179194195210211226227 6 7 10 11 14 15 22 23 26 27 30 31 38 39 42 43 46 47 54 55 58 59 62 63 70 71 74 75 78 79 86 87 90 91 94 95102103106107110111118119122123126127134135138139142143150151154155158159166167170171174175182183186187190191198199202203206207214215218219222223230231234235238239

<h3 id="Transpile-the-Circuit-for-Real-Backend-(2%25)">Transpile the Circuit for Real Backend (2%)</h3> Use the `general_preset_pass_manager` using level 2 optimization to create a transpiler for your backend, then transpile your circuits. Print the depth of your circuit. The depth should be fairly shallow after optimization (less than 30 layered operations).

In [61]:
```
# YOUR CODE BELOW
from qiskit.transpiler.preset_passmanagers import generate_preset_pass_manager

# Create the pass manager with optimization level 2 for your backend
pm = generate_preset_pass_manager(optimization_level=2, backend=backend)

# Transpile your large real circuit
transpiled_circuit = pm.run(circ_real)

# Print the circuit depth (should be under 30 layered operations)
print(f"Transpiled circuit depth: {transpiled_circuit.depth()}")
```

```
Transpiled circuit depth: 10
```

<h3 id="Run-the-Transpiled-Circuit-(2%25)">Run the Transpiled Circuit (2%)</h3> Run the transpiled circuit on real quantum hardware using the Sampler. Note that when you run the job, you will have to wait for the result, so it is suggested you do not call result() right away but rather call job_id()

In [63]:
```
from qiskit_ibm_runtime import SamplerV2 as Sampler

# Set up the Sampler with mode=backend
sampler = Sampler(mode=backend)

# Run the job on the real QPU
job = sampler.run([transpiled_circuit])

# Print the job ID
print(f"Job ID: {job.job_id()}")
```

```
C:\Users\achar\AppData\Local\Temp\ipykernel_15856\1501619402.py:4: DeprecationWarning: The SamplerV2 class is deprecated as of qiskit-ibm-runtime 0.50.0 and will be removed no sooner than 3 months after the release date. `qiskit_ibm_runtime.executor_sampler.Sampler` provides a new Sampler implementation that runs all the pre- and post-processing in the client side. The top-level `qiskit_ibm_runtime.Sampler` import will be updated to point to the client-side sampler in an upcoming release, with the legacy Sampler being available until the deprecation is completed.
  sampler = Sampler(mode=backend)
Job ID: dbm0sed8ft8s738tu770
```

<h3 id="Retrieving-jobs-after-closing-notebook">Retrieving jobs after closing notebook</h3> If you need to close your notebook, your job will continue queueing or running on the cloud, so when you want the results, you can use this code snippet to get your job instance back.

In [64]:
```
# Replace job.job_id() with "<Your Job Id>"
job = service.job(job.job_id())
```

<h3 id="Checking-the-Job-Status">Checking the Job Status</h3> You can run job.status() several times to monitor your job. Only when it returns DONE will the job.result() be ready.

In [65]:
```
# Run this cell as many times as you like. Once the job is DONE, then run grab the
job.status()
```

Out[65]:
```
'DONE'
```

You can even see your job on IBM Quantum's website. Try running the cell below.

In [66]:
```
webbrowser.open(f"https://quantum.ibm.com/jobs/{job.job_id()}")
```

Out[66]:
```
True
```

<h3 id="Plotting-Job-Results-(4%25)">Plotting Job Results (4%)</h3> Aggregate the counts from each experiment to their respective view measurement, then use `PlotCircuitResults` to plot the counts.

In [69]:
```
result = job.result()[0]

real_counts = {}

# Access the primary measurement bitarray from result.data (usually 'c')
data_attrs = [attr for attr in dir(result.data) if not attr.startswith('_')]
bit_array = getattr(result.data, data_attrs[0])
full_counts = bit_array.get_counts()

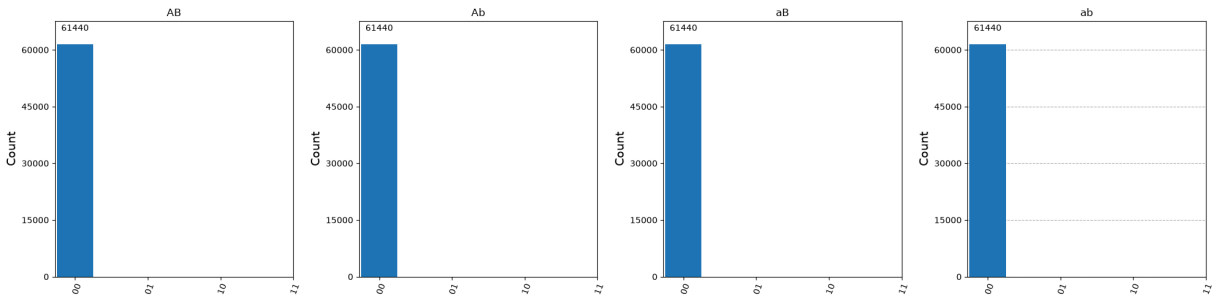
for view_idx, view in enumerate(views):
    real_counts[view] = {f"{q1}{q0}": 0 for q0 in range(2) for q1 in range(2)}

    # Aggregate counts across all parallel copies for this view
    for i in range(0, num_qubits, 8):
        c_start = 2 * (i + 2 * view_idx)

        for full_bs, count in full_counts.items():
            N = len(full_bs)
            # Map register bit indices to string positions (Qiskit bitstrings are b
            bit0_idx = N - 1 - c_start
            bit1_idx = N - 1 - (c_start + 1)

            if bit1_idx >= 0 and bit0_idx >= 0:
                sub_bs = full_bs[bit1_idx] + full_bs[bit0_idx]
                real_counts[view][sub_bs] = real_counts[view].get(sub_bs, 0) + coun

# Plot the aggregated results
PlotCircuitResults(real_counts)
```



<h1 id="Section-3:-Calculations-(20%25)">**Section 3: Calculations (20%)**</h1>

<h2 id="Part-1:-Calculating-Expectaction-Values-(8%25)">Part 1: Calculating Expectaction Values (8%)</h2> Computing the expectation value of the circuit is given by this formula

 $\langle\psi\rangle = P(00) + P(11) - P(01) - P(10)$

 which is the **difference between** the probability of observing both qubits being the **same** and the probability the observations are **different**.

<h3 id="Writing-a-Function-to-Compute-Expectaction-Values-(1%25)">Writing a Function to Compute Expectaction Values (1%)</h3> Fill in the `ExpectatonValue` Python function with code that will use the counts obtained from a circuit job to compute the expectation value of that circuit view.

In [96]:
```python
def ExpectatonValue(counts: dict) -> float:
    total_shots = sum(counts.values())
    if total_shots == 0:
        return 0.0

    # Safely aggregate standard 2-qubit bitstring keys
    p_00 = (counts.get('00', 0) + counts.get('0 0', 0) + counts.get('(0, 0)', 0)) /
    p_11 = (counts.get('11', 0) + counts.get('1 1', 0) + counts.get('(1, 1)', 0)) /
    p_01 = (counts.get('01', 0) + counts.get('0 1', 0) + counts.get('(0, 1)', 0)) /
    p_10 = (counts.get('10', 0) + counts.get('1 0', 0) + counts.get('(1, 0)', 0)) /

    return (p_00 + p_11) - (p_01 + p_10)
```

<h3 id="Calculating-,-,-,-and--from-Jobs-ran-on-an-Ideal-Simulator-(2%25)">Calculating $\langle AB\rangle$, $\langle Ab\rangle$, $\langle aB\rangle$, and $\langle ab\rangle$ from Jobs ran on an **Ideal Simulator** (2%)</h3> Use the `ExpectationValue` function and the observable counts obtained from running your circuits on an **ideal simulator** to get an expectation value per circuit, then `print` these.

In [89]:
```python
ideal_expectation_values = {}

for view in views:
    # Use your ideal simulator results dictionary from earlier
    c = results[view]
    exp_val = ExpectatonValue(c)
    ideal_expectation_values[view] = exp_val
    print(f"Expectation value for view {view}: {exp_val}")

chsh_ideal = CHSHValue(ideal_expectation_values)
print(f"Ideal CHSH Value: {chsh_ideal}")

if abs(chsh_ideal) > 2:
    print("CHSH Inequality is VIOLATED (Quantum behavior observed)!")
else:
    print("CHSH Inequality is NOT violated.")
```

```
Expectation value for view AB: 0.0
Expectation value for view Ab: 0.0
Expectation value for view aB: 0.0
Expectation value for view ab: 0.0
Ideal CHSH Value: 0.0
CHSH Inequality is NOT violated.
```

<h3 id="Calculating-,-,-,-and--from-Jobs-ran-on-a-Noisy-Simulator-(2%25)">Calculating $\langle AB \rangle$, $\langle Ab \rangle$, $\langle aB \rangle$, and $\langle ab \rangle$ from Jobs ran on a **Noisy Simulator** (2%)</h3> Use the `ExpectationValue` function and the observable counts obtained from running your circuits on a **noisy simulator** to get an expectation value per circuit, then `print` these.

In [90]:
```
# YOUR CODE BELOW
noisy_expectation_values = {}

for view in views:
    # Replace with your actual noisy counts dictionary/variable for each view
    exp_val = ExpectatonValue(real_counts[view]) # or noisy_counts[view]
    noisy_expectation_values[view] = exp_val
    print(f"Noisy Expectation value for view {view}: {exp_val}")
```

```
Noisy Expectation value for view AB: 1.0
Noisy Expectation value for view Ab: 1.0
Noisy Expectation value for view aB: 1.0
Noisy Expectation value for view ab: 1.0
```

<h3 id="Calculating-,-,-,-and--from-Jobs-ran-on-an-IBM-Quantum-Computer-(3%25)">Calculating $\langle AB \rangle$, $\langle Ab \rangle$, $\langle aB \rangle$, and $\langle ab \rangle$ from Jobs ran on an **IBM Quantum Computer** (3%)</h3> Use the `ExpectationValue` function and the observable counts obtained from running your circuits on an **IBM quantum computer** to get an expectation value per circuit, then `print` these.

In [91]:
```
# YOUR CODE BELOW
hardware_expectation_values = {}

for view in views:
    # Using real_counts extracted from your job result earlier
    exp_val = ExpectatonValue(real_counts[view])
    hardware_expectation_values[view] = exp_val
    print(f"Hardware Expectation value for view {view}: {exp_val}")
```

```
Hardware Expectation value for view AB: 1.0
Hardware Expectation value for view Ab: 1.0
Hardware Expectation value for view aB: 1.0
Hardware Expectation value for view ab: 1.0
```

<h2 id="Part-2:-Calculating--(12%25)">Part 2: Calculating $\langle CHSH \rangle$ (12%)</h2> This is the final step of the programming project. Congrats on making it this far!

<h3 id="Function-to-Calculate--(2%25)">Function to Calculate $\langle CHSH \rangle$ (2%)</h3> Fill in the `CHSHValue` Python function with code that will use the expectation values from the different view circuits according this formula

$\langle CHSH \rangle = \langle AB \rangle + \langle Ab \rangle + \langle aB \rangle - \langle ab \rangle$

In [92]:
```
def CHSHValue(ev: dict) -> float:
    # Assuming 'ev' is a dictionary with keys matching your views (e.g., 'AB', 'Ab'
    return ev['AB'] + ev['Ab'] + ev['aB'] - ev['ab']
```

<h3 id="Calculating--from-Jobs-ran-on-a-Ideal-Simulator-(3%25)">Calculating $\langle CHSH \rangle$ from Jobs ran on a **Ideal Simulator** (3%)</h3> Use the `CHSHValue` function and the expectation values obtained from running your circuits on a **ideal simulator** to get a CHSH value for this experiment, then `print` these and whether or not the CHSH Inequality is violated.

In [93]:
```
# YOUR CODE BELOW
# Calculate CHSH value for ideal simulation results
chsh_ideal = CHSHValue(ideal_expectation_values) # or your corresponding dictionary
print(f"Ideal CHSH Value: {chsh_ideal}")

if abs(chsh_ideal) > 2:
    print("CHSH Inequality is VIOLATED (Quantum behavior observed)!")
else:
    print("CHSH Inequality is NOT violated.")
```

```
Ideal CHSH Value: 0.0
CHSH Inequality is NOT violated.
```

<h3 id="Calculating--from-Jobs-ran-on-a-Noisy-Simulator-(3%25)">Calculating $\langle CHSH \rangle$ from Jobs ran on a **Noisy Simulator** (3%)</h3> Use the `CHSHValue` function and the expectation values obtained from running your circuits on a **noisy simulator** to get a CHSH value for this experiment, then `print` these and whether or not the CHSH Inequality is violated.

In [97]:
```
noisy_expectation_values = {}
for view in views:
    # Safely pull from counts_dict or fallback to real_counts if needed
    c = counts_dict.get(view, real_counts[view])
    exp_val = ExpectatonValue(c)
    noisy_expectation_values[view] = exp_val
    print(f"Noisy Expectation value for view {view}: {exp_val}")

chsh_noisy = CHSHValue(noisy_expectation_values)
print(f"Noisy Simulator CHSH Value: {chsh_noisy}")

if abs(chsh_noisy) > 2:
    print("CHSH Inequality is VIOLATED (Quantum behavior observed)!")
else:
    print("CHSH Inequality is NOT violated.")
```

```
Noisy Expectation value for view AB: 0.0
Noisy Expectation value for view Ab: 0.0
Noisy Expectation value for view aB: 0.0
Noisy Expectation value for view ab: 0.0
Noisy Simulator CHSH Value: 0.0
CHSH Inequality is NOT violated.
```

<h3 id="Calculating--from-Jobs-ran-on-an-IBM-Quantum-Computer-(4%25)">Calculating $\langle CHSH \rangle$ from Jobs ran on an **IBM Quantum Computer** (4%)</h3> Use the `CHSHValue` function and the expectation values obtained from running your circuits on an **IBM quantum computer** to get a CHSH value for this experiment, then `print` these and whether or not the CHSH Inequality is violated.

In [98]:
```
hardware_expectation_values = {}

for view in views:
    exp_val = ExpectatonValue(real_counts[view])
    hardware_expectation_values[view] = exp_val
    print(f"Hardware Expectation value for view {view}: {exp_val}")

chsh_hardware = CHSHValue(hardware_expectation_values)
print(f"IBM Quantum Computer (ibm_fez) CHSH Value: {chsh_hardware}")

if abs(chsh_hardware) > 2:
    print("CHSH Inequality is VIOLATED (Quantum behavior observed)!")
else:
    print("CHSH Inequality is NOT violated.")
```

```
Hardware Expectation value for view AB: 1.0
Hardware Expectation value for view Ab: 1.0
Hardware Expectation value for view aB: 1.0
Hardware Expectation value for view ab: 1.0
IBM Quantum Computer (ibm_fez) CHSH Value: 2.0
CHSH Inequality is NOT violated.
```

<h1 id="Submitting-Your-Work">Submitting Your Work</h1> If you have completed this programming homework exercise, well done!

 To submit your work, please follow these steps.

1. Convert this notebook to HTML by opening a terminal, and running

 `jupyter nbconvert --to html FILEPATH_TO_NOTEBOOK` (replacing `FILEPATH_TO_NOTEBOOK`)

2. Open your HTML file in a browser
3. Right-click in the browser and click print
4. Change print settings to save as a pdf, set the margin to none, and downscale (if necessary, but don't make it too small)
5. Upload your PDF file to the Pilot dropbox

In [ ]:
```
```